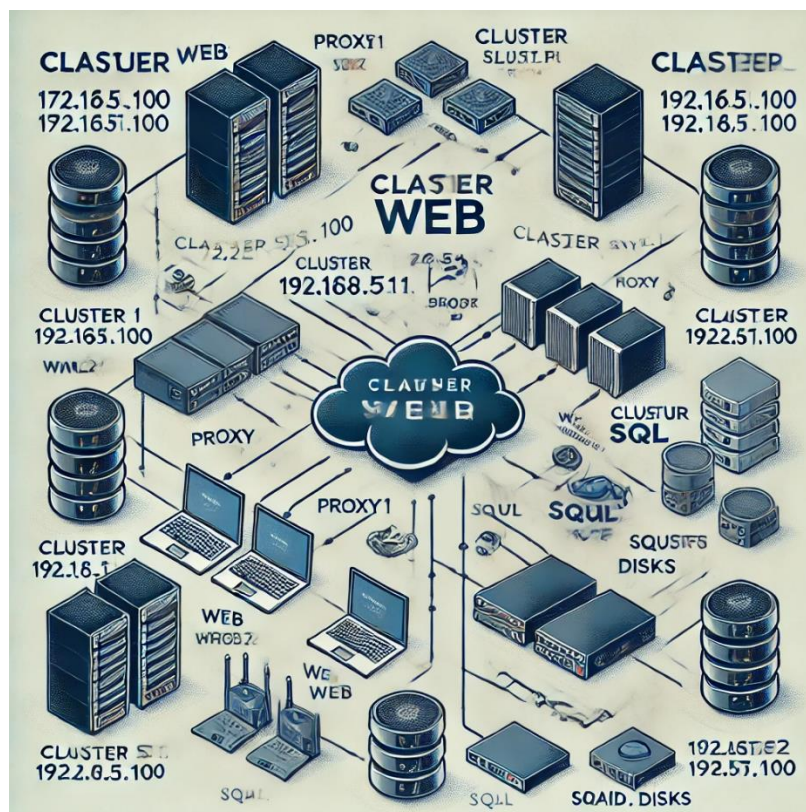


**LICENCIATURA EM
SEGURANÇA INFORMÁTICA EM REDES DE COMPUTADORES**

SISTEMAS CRÍTICOS

Trabalho Prático



David Santos – 8220651

ESTG, dezembro de 2024

ÍNDICE

1	Introdução.....	1
2	Desenho da Infraestrutura	2
2.1	Representação Visual.....	2
2.2	Visão geral.....	2
3	Criação da Infraestrutura	4
3.1	Decisões e Justificativas.....	4
3.2	Configuração com Vagrant	4
4	Configuração e Implementação dos Objetivos.....	7
4.1	Configurações Iniciais	7
4.2	Máquinas Web1 e Web2	8
4.3	Máquinas SQL1 e SQL2	10
4.4	Sistema de Balanceamento de Carga	13
4.5	Sistema de Ficheiros Distribuído	15
4.6	Configuração da Base de Dados para Integração com os Web	16
5	Sites da empresa.....	17
5.1	Cacti	17
5.2	Ganglia	21
6	Avaliação da Infraestrutura	24
6.1	Impacto para Serviço Web.....	24
6.2	Impacto para Serviço SQL	29
7	Conclusão.....	32

1 INTRODUÇÃO

Em sistemas críticos, como os utilizados em ambientes empresariais e de infraestrutura de TI, a alta disponibilidade (HA) é essencial para garantir que os serviços permaneçam operacionais mesmo diante de falhas de hardware ou software. Esses sistemas devem ser projetados para resistir a interrupções, minimizando o impacto nas operações e assegurando continuidade. Em um cenário ideal, a alta disponibilidade não só reduz o tempo de inatividade, mas também distribui cargas de trabalho de forma eficiente e segura, contribuindo para o desempenho e a resiliência.

Este trabalho tem como objetivo a criação de uma infraestrutura de TI altamente disponível e escalável, baseada em balanceamento de carga, redundância de servidores e armazenamento compartilhado. Através da utilização de ferramentas como HAProxy, MariaDB (com replicação ativa/passiva) e GlusterFS, buscamos implementar uma solução robusta que suporte falhas em qualquer ponto da infraestrutura, mantendo os serviços e dados acessíveis aos utilizadores sem interrupções significativas.

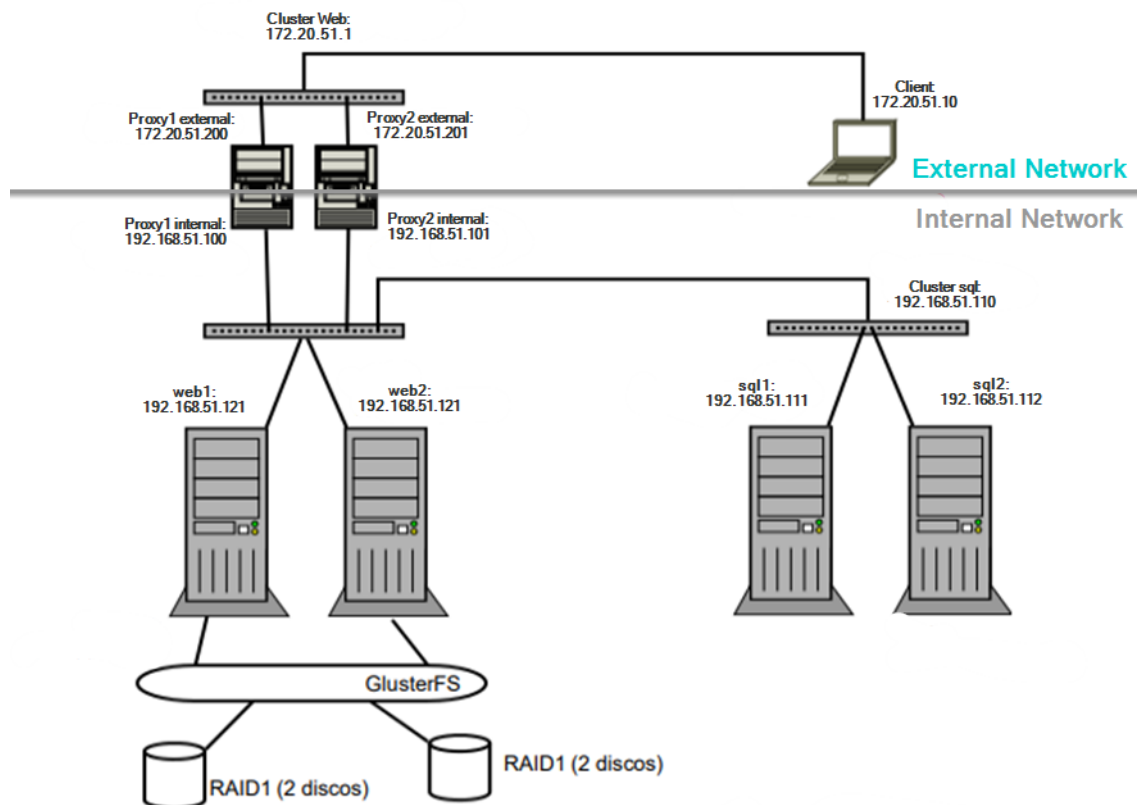
Os objetivos específicos deste trabalho são:

- Configurar um ambiente com dois servidores web e dois servidores de base de dados em alta disponibilidade.
- Implementar balanceamento de carga para distribuir as requisições entre os servidores web.
- Garantir a redundância e tolerância a falhas utilizando sistemas de arquivos compartilhados e replicação de base de dados.
- Realizar testes de desempenho e avaliar a eficiência da infraestrutura sob diferentes condições de carga.
- Propor soluções e melhorias para otimizar a escalabilidade e a resiliência da arquitetura proposta.

Este trabalho visa demonstrar como a combinação de várias tecnologias de alta disponibilidade pode ser aplicada para criar uma infraestrutura confiável e eficiente, atendendo aos desafios impostos pelos sistemas críticos em ambientes modernos.

2 DESENHO DA INFRAESTRUTURA

2.1 Representação Visual



2.2 Visão geral

A infraestrutura foi desenvolvida para garantir alta disponibilidade, tolerância a falhas e balanceamento de carga, conforme os requisitos do projeto. Abaixo está uma visão geral da estrutura criada:

2.2.1 Componentes Principais

1. Servidores Web (web1 e web2):

- Configurados em balanceamento de carga utilizando **HAProxy**.
- Utilizam um sistema de armazenamento redundante em **RAID 1**, com volumes compartilhados via **GlusterFS**.
- O diretório principal para os ficheiros web está localizado em `/cluster/www/`.

2. Servidores de Base de Dados (sql1 e sql2):

- Executam o serviço **MariaDB** em modo ativo/passivo, garantindo alta disponibilidade.
- Os dados da base de dados são armazenados em /cluster/sql/, um volume compartilhado configurado com GlusterFS.

3. Proxies de Balanceamento (haproxy1 e haproxy2):

- Configurados em modo ativo/passivo, para garantir redundância e tolerância a falhas no balanceamento de carga.

4. Sistema de Ficheiros Distribuído:

- Implementado nos servidores web (web1 e web2) usando **GlusterFS**.
- Volumes criados:
 - storage-www para ficheiros da aplicação web.
 - storage-sql para dados da base de dados.

5. Cliente (client):

- Máquina utilizada para executar testes e simular tráfego para a infraestrutura.

2.2.2 Conexões e Comunicação

• Rede Interna Privada:

- Range: 192.168.51.0/24.
- Conecta os servidores web, proxies e base de dados, com endereços IP fixos para previsibilidade e integração eficiente.
- O tráfego entre os servidores web e base de dados ativo é roteado por esta rede, com acesso aos volumes compartilhados via GlusterFS.

• Simulação de Rede Externa:

- Range: 172.20.51.0/24.
- Usada para comunicação entre os clientes e os proxies (via endereço virtual do cluster). O HAProxy distribui as requisições dos clientes para os servidores web ativos pela rede interna.

• Alta Disponibilidade e Tolerância a falhas:

- Caso um servidor ou proxy falhe, os serviços são automaticamente redistribuídos pelo cluster ativo/passivo, garantindo continuidade.

• Armazenamento Compartilhado:

- O sistema de ficheiros GlusterFS permite o compartilhamento de volumes entre servidores, garantindo consistência nos dados armazenados.

3 CRIAÇÃO DA INFRAESTRUTURA

Para a criação da infraestrutura, optei por utilizar o **Vagrant** em conjunto com **scripts Bash** como forma de automatizar tarefas essenciais. Isso incluiu a criação de máquinas virtuais, atribuição de endereços IP, instalação e configuração de serviços, além da preparação geral do ambiente. Meu objetivo foi simplificar o processo e reduzir a necessidade de intervenção manual, permitindo que o ambiente estivesse pronto para uso com um simples comando **vagrant up**.

Todos os ficheiros, incluindo os scripts e configurações auxiliares usados para cumprir os objetivos do trabalho, serão disponibilizados juntamente com o relatório. Ao longo desta seção, apresento trechos de scripts e configurações que considero relevantes, destacando as decisões tomadas e como estas contribuíram para atender aos requisitos do trabalho.

3.1 Decisões e Justificativas

Escolhi o **Vagrant** devido à sua praticidade na criação de ambientes consistentes e reproduzíveis. Com o suporte ao provisionamento automatizado, consegui preparar as máquinas quase que completamente desde o início. Para complementar, desenvolvi **scripts Bash** que realizam verificações cuidadosas para evitar sobrescrever ou danificar configurações já existentes. Dessa forma, as máquinas se aproximam ao máximo do cenário final esperado, com automação integrada diretamente ao provisionamento do Vagrant.

Além disso, dediquei especial atenção à clareza e documentação dos scripts. A maioria dos comandos inclui comentários explicativos, e usei prints (echo) nos pontos principais para fornecer feedback visual durante a execução. Essa abordagem facilita tanto a compreensão do processo quanto a análise de eventuais problemas.

3.2 Configuração com Vagrant

No **Vagrantfile**, organizei a definição das máquinas de forma clara e eficiente. Aqui estão algumas das configurações implementadas:

- Máquinas definidas: web1, web2, haproxy1, haproxy2, sql1, sql2 e client.
- Sincronização de ficheiros: A pasta `setup_files` contém todos os scripts de configuração e auxiliares, acessíveis por todas as máquinas virtuais.
- Discos adicionais: Adicionei dois discos de 3GB para cada máquina web para suportar o sistema de armazenamento distribuído GlusterFS em RAID 1.

- Endereços IP estáticos: Cada máquina recebeu endereços IP fixos, garantindo previsibilidade e consistência na configuração do cluster.
- Provisionamento automatizado: Scripts Bash são executados automaticamente durante a criação das máquinas, configurando serviços essenciais como RAID, GlusterFS, HAProxy e MariaDB.

```
Vagrantfile X
sc-workspace > Vagrantfile
You, 2 minutes ago | 1 author (You)
1 Vagrant.configure("2") do |config|
2
3   > config.vm.define "web1" do |web1| ...
25   end
26
27   > config.vm.define "web2" do |web2| ...
49   end
50
51   > config.vm.define "haproxy1" do |haproxy1| ...
72   end
73
74   > config.vm.define "haproxy2" do |haproxy2| ...
94   end
95
96   > config.vm.define "sql1" do |sql1| ...
116   end
117
118   > config.vm.define "sql2" do |sql2| ...
138   end
139
140   > config.vm.define "client" do |client| ...
155   end
156
157   end
158
```

3.2.1 Máquinas WEB

<pre>config.vm.define "web1" do web1 web1.vm.box = "ubuntu/focal64" web1.vm.network "private_network", type: "static", ip: "192.168.51.121" web1.vm.hostname = "web1" config.vm.synced_folder "../setup_files/", "/vagrant" web1.vm.disk :disk, size: "3GB", name: "sdc" web1.vm.disk :disk, size: "3GB", name: "sdd" web1.vm.provider "virtualbox" do vb vb.customize ["modifyvm", :id, "--groups", "/ClusterSC"] end web1.vm.provision "shell", inline: <<-SHELL chmod +x /vagrant/*.sh bash /vagrant/first.sh bash /vagrant/hosts.sh bash /vagrant/raid.sh bash /vagrant/glusterfs.sh bash /vagrant/webserver.sh sudo reboot SHELL end</pre>	<pre>config.vm.define "web2" do web2 web2.vm.box = "ubuntu/focal64" web2.vm.network "private_network", type: "static", ip: "192.168.51.122" web2.vm.hostname = "web2" config.vm.synced_folder "../setup_files/", "/vagrant" web2.vm.disk :disk, size: "3GB", name: "sdc" web2.vm.disk :disk, size: "3GB", name: "sdd" web2.vm.provider "virtualbox" do vb vb.customize ["modifyvm", :id, "--groups", "/ClusterSC"] end web2.vm.provision "shell", inline: <<-SHELL chmod +x /vagrant/*.sh bash /vagrant/first.sh bash /vagrant/hosts.sh bash /vagrant/raid.sh bash /vagrant/glusterfs.sh bash /vagrant/webserver.sh sudo reboot SHELL end</pre>
--	--

3.2.2 Máquinas Proxy(HA)

```
config.vm.define "haproxy1" do |haproxy1|
  haproxy1.vm.box = "ubuntu/focal64"
  haproxy1.vm.network "private_network", type: "static", ip: "172.20.51.200"
  haproxy1.vm.network "private_network", type: "static", ip: "192.168.51.100"
  haproxy1.vm.hostname = "haproxy1"
  config.vm.synced_folder "setup_files/", "/vagrant"

  haproxy1.vm.provider "virtualbox" do |vb|
    vb.customize ["modifyvm", :id, "--groups", "/ClusterSC"]
    vb.customize ["modifyvm", :id, "--nicpromisc2", "allow-all"]
  end

  haproxy1.vm.provision "shell", inline: <<-SHELL
    chmod +x /vagrant/*.sh
    bash /vagrant/first.sh
    bash /vagrant/hosts.sh
    bash /vagrant/ha_proxy.sh
    bash /vagrant/cluster_proxy.sh
    sudo reboot
  SHELL
end

config.vm.define "haproxy2" do |haproxy2|
  haproxy2.vm.box = "ubuntu/focal64"
  haproxy2.vm.network "private_network", type: "static", ip: "172.20.51.201"
  haproxy2.vm.network "private_network", type: "static", ip: "192.168.51.101"
  haproxy2.vm.hostname = "haproxy2"
  config.vm.synced_folder "setup_files/", "/vagrant"

  haproxy2.vm.provider "virtualbox" do |vb|
    vb.customize ["modifyvm", :id, "--groups", "/ClusterSC"]
    vb.customize ["modifyvm", :id, "--nicpromisc2", "allow-all"]
  end

  haproxy2.vm.provision "shell", inline: <<-SHELL
    chmod +x /vagrant/*.sh
    bash /vagrant/first.sh
    bash /vagrant/hosts.sh
    bash /vagrant/ha_proxy.sh
    bash /vagrant/cluster_proxy.sh
    sudo reboot
  SHELL
end
```

3.2.3 Máquinas SQL

```
config.vm.define "sql1" do |sql1|
  sql1.vm.box = "ubuntu/focal64"
  sql1.vm.network "private_network", type: "static", ip: "192.168.51.111"
  sql1.vm.hostname = "sql1"
  config.vm.synced_folder "setup_files/", "/vagrant"

  sql1.vm.provider "virtualbox" do |vb|
    vb.customize ["modifyvm", :id, "--groups", "/ClusterSC"]
    vb.customize ["modifyvm", :id, "--nicpromisc2", "allow-all"]
  end

  sql1.vm.provision "shell", inline: <<-SHELL
    chmod +x /vagrant/*.sh
    bash /vagrant/first.sh
    bash /vagrant/hosts.sh
    bash /vagrant/glusterfs.sh
    bash /vagrant/database.sh
    bash /vagrant/cluster_database.sh
    sudo reboot
  SHELL
end

config.vm.define "sql2" do |sql2|
  sql2.vm.box = "ubuntu/focal64"
  sql2.vm.network "private_network", type: "static", ip: "192.168.51.112"
  sql2.vm.hostname = "sql2"
  config.vm.synced_folder "setup_files/", "/vagrant"

  sql2.vm.provider "virtualbox" do |vb|
    vb.customize ["modifyvm", :id, "--groups", "/ClusterSC"]
    vb.customize ["modifyvm", :id, "--nicpromisc2", "allow-all"]
  end

  sql2.vm.provision "shell", inline: <<-SHELL
    chmod +x /vagrant/*.sh
    bash /vagrant/first.sh
    bash /vagrant/hosts.sh
    bash /vagrant/glusterfs.sh
    bash /vagrant/database.sh
    bash /vagrant/cluster_database.sh
    sudo reboot
  SHELL
end
```

3.2.4 Máquina Cliente

```
config.vm.define "client" do |client|
  client.vm.box = "ubuntu/focal64"
  client.vm.network "private_network", type: "static", ip: "172.20.51.10"
  client.vm.hostname = "client"
  config.vm.synced_folder "setup_files/", "/vagrant"

  client.vm.provider "virtualbox" do |vb|
    vb.customize ["modifyvm", :id, "--groups", "/ClusterSC"]
  end

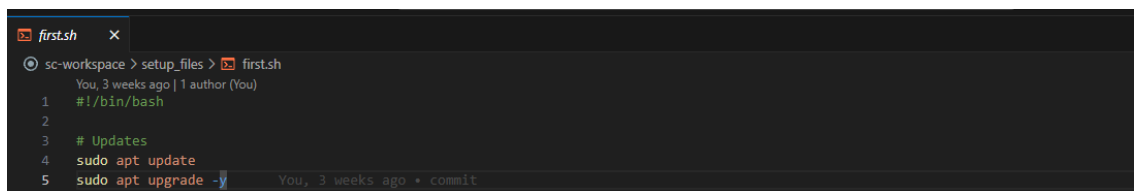
  client.vm.provision "shell", inline: <<-SHELL
    sudo apt-get update
    sudo apt-get install -y ubuntu-desktop
    sudo reboot
  SHELL
end
```


4 CONFIGURAÇÃO E IMPLEMENTAÇÃO DOS OBJETIVOS

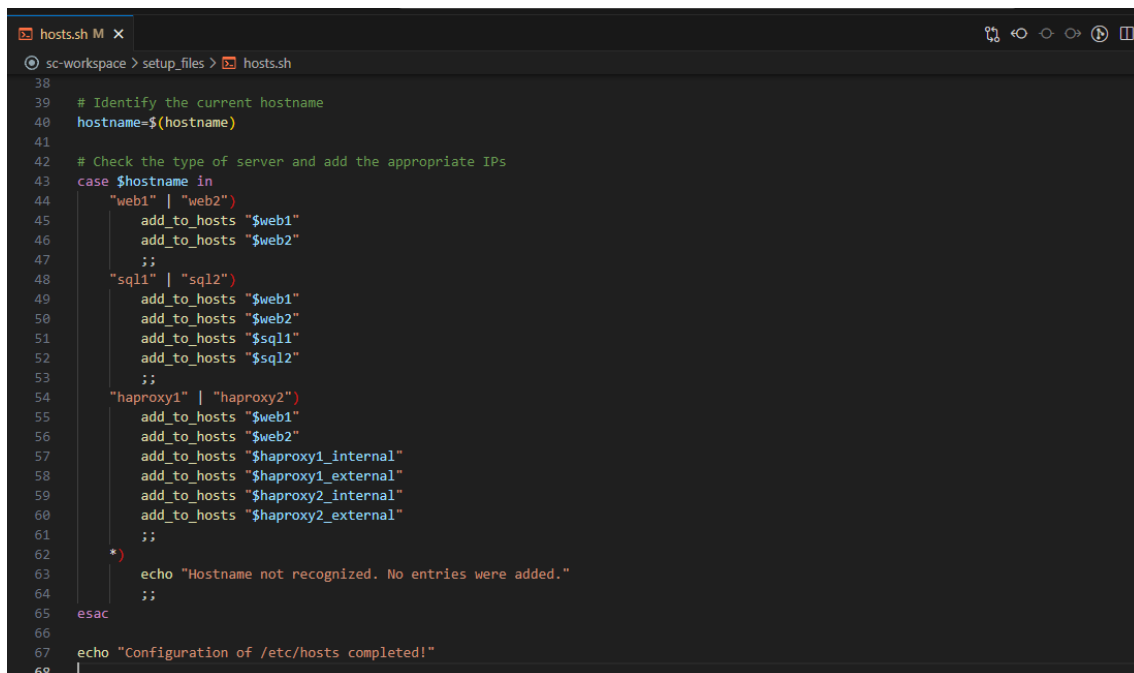
Nesta seção, detalho os scripts desenvolvidos e como foram utilizados para atender aos principais objetivos do trabalho. Cada conjunto de scripts foi criado para configurar, validar e implementar os serviços de forma automática, garantindo que os requisitos fossem cumpridos de forma eficiente.

4.1 Configurações Iniciais

Os seguintes scripts tratam-se de códigos que foram executados em todas as máquinas da infraestrutura. Estes têm como objetivo uma preparação inicial das máquinas, atualizando-as e assegurando que estas têm conhecimento dos nomes(hosts) das máquinas com as quais irá comunicar por meio de alguma configuração. Isso otimiza comandos posteriormente executados tendo em conta que deixa de ser necessário a especificação dos IPs associados.



```
first.sh X
sc-workspace > setup_files > first.sh
You, 3 weeks ago | 1 author (You)
1  #!/bin/bash
2
3  # Updates
4  sudo apt update
5  sudo apt upgrade -y You, 3 weeks ago • commit
```



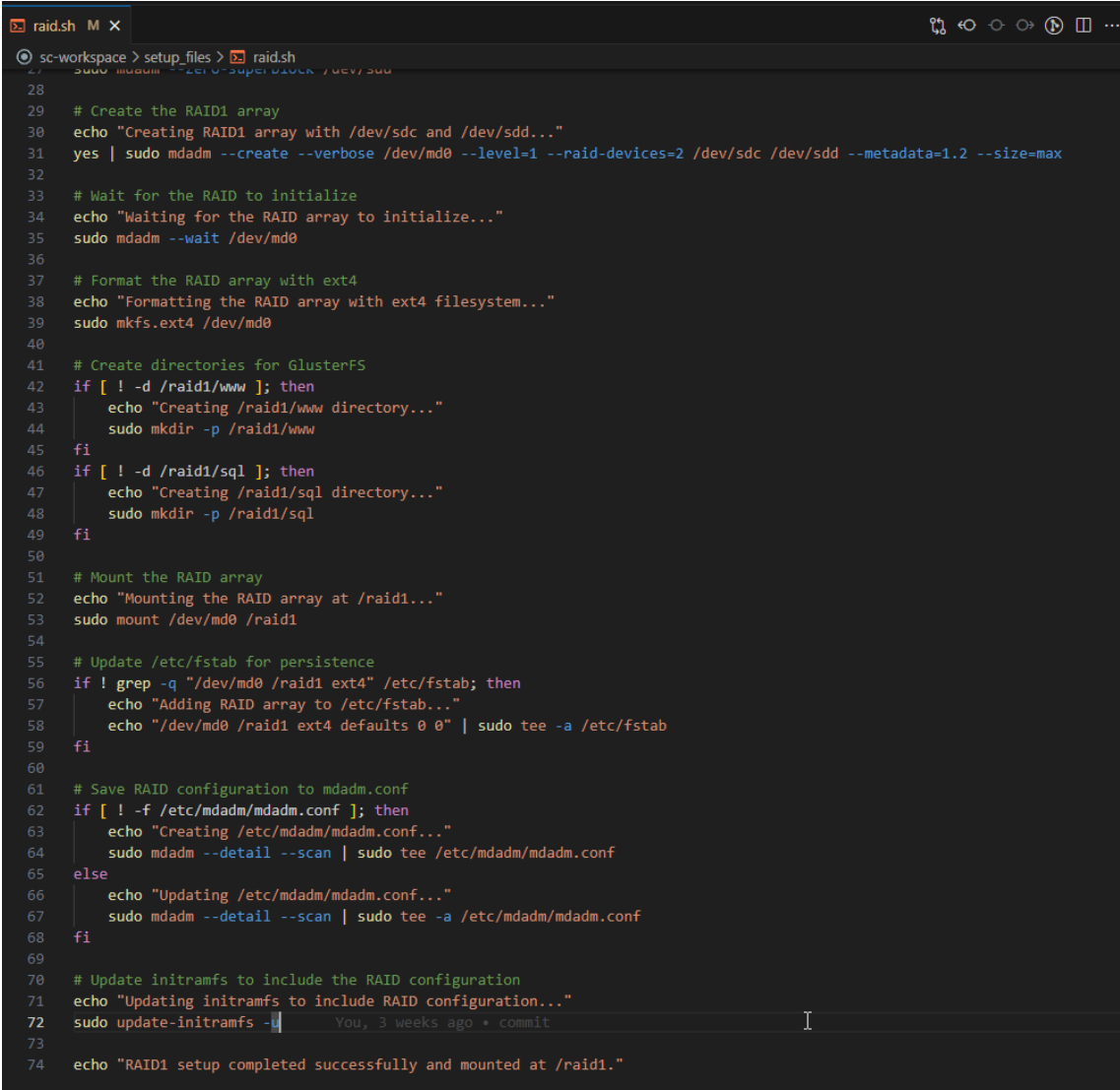
```
hosts.sh M X
sc-workspace > setup_files > hosts.sh
38
39 # Identify the current hostname
40 hostname=$(hostname)
41
42 # Check the type of server and add the appropriate IPs
43 case $hostname in
44     "web1" | "web2")
45         add_to_hosts "$web1"
46         add_to_hosts "$web2"
47         ;;
48     "sql1" | "sql2")
49         add_to_hosts "$web1"
50         add_to_hosts "$web2"
51         add_to_hosts "$sql1"
52         add_to_hosts "$sql2"
53         ;;
54     "haproxy1" | "haproxy2")
55         add_to_hosts "$web1"
56         add_to_hosts "$web2"
57         add_to_hosts "$haproxy1_internal"
58         add_to_hosts "$haproxy1_external"
59         add_to_hosts "$haproxy2_internal"
60         add_to_hosts "$haproxy2_external"
61         ;;
62     *)
63         echo "Hostname not recognized. No entries were added."
64         ;;
65 esac
66
67 echo "Configuration of /etc/hosts completed!"
68
```

4.2 Máquinas Web1 e Web2

Os servidores web1 e web2 foram configurados para operar em balanceamento de carga, utilizando RAID 1 para armazenamento redundante, GlusterFS para compartilhamento de ficheiros, e Nginx como servidor web. Os seguintes scripts foram utilizados:

4.2.1 *raid.sh*

Configura dois discos de 3GB em RAID 1, monta o volume em `/raid1` e cria os diretórios necessários para os dados.



```
sc-workspace > setup_files > raid.sh
28
29 # Create the RAID1 array
30 echo "Creating RAID1 array with /dev/sdc and /dev/sdd..."
31 yes | sudo mdadm --create --verbose /dev/md0 --level=1 --raid-devices=2 /dev/sdc /dev/sdd --metadata=1.2 --size=max
32
33 # Wait for the RAID to initialize
34 echo "Waiting for the RAID array to initialize..."
35 sudo mdadm --wait /dev/md0
36
37 # Format the RAID array with ext4
38 echo "Formatting the RAID array with ext4 filesystem..."
39 sudo mkfs.ext4 /dev/md0
40
41 # Create directories for GlusterFS
42 if [ ! -d /raid1/www ]; then
43     echo "Creating /raid1/www directory..."
44     sudo mkdir -p /raid1/www
45 fi
46 if [ ! -d /raid1/sql ]; then
47     echo "Creating /raid1/sql directory..."
48     sudo mkdir -p /raid1/sql
49 fi
50
51 # Mount the RAID array
52 echo "Mounting the RAID array at /raid1..."
53 sudo mount /dev/md0 /raid1
54
55 # Update /etc/fstab for persistence
56 if ! grep -q "/dev/md0 /raid1 ext4" /etc/fstab; then
57     echo "Adding RAID array to /etc/fstab..."
58     echo "/dev/md0 /raid1 ext4 defaults 0 0" | sudo tee -a /etc/fstab
59 fi
60
61 # Save RAID configuration to mdadm.conf
62 if [ ! -f /etc/mdadm/mdadm.conf ]; then
63     echo "Creating /etc/mdadm/mdadm.conf..."
64     sudo mdadm --detail --scan | sudo tee /etc/mdadm/mdadm.conf
65 else
66     echo "Updating /etc/mdadm/mdadm.conf..."
67     sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
68 fi
69
70 # Update initramfs to include the RAID configuration
71 echo "Updating initramfs to include RAID configuration..."
72 sudo update-initramfs -u You, 3 weeks ago * commit
73
74 echo "RAID1 setup completed successfully and mounted at /raid1."
```

4.2.2 gluster.sh

Configura o GlusterFS para replicar os dados entre web1 e web2. Cria o volume storage-www e monta-o em /cluster/www. Também assegura a criação do volume para armazenamento dos SQL.

```
glusterfs.sh X
sc-workspace > setup_files > glusterfs.sh
3  set -e # Exit script on any error
4
5  # Install GlusterFS
6  echo "Installing GlusterFS..."
7  sudo apt-get install -y glusterfs-server glusterfs-client
8
9  # Start and enable the GlusterFS service
10 sudo systemctl start glusterd
11 sudo systemctl enable glusterd
12
```

```
glusterfs.sh X
sc-workspace > setup_files > glusterfs.sh
17 if [[ "$HOSTNAME" == "web1" || "$HOSTNAME" == "web2" ]]; then
18     echo "Configuring GlusterFS on web node ($HOSTNAME)..."
19
20     # Ensure /raid1 exists and is mounted (RAID1 should already be configured)
21     if ! mount | grep -q "/raid1"; then
22         echo "RAID1 is not mounted on /raid1. Please ensure raid.sh is executed first."
23         exit 1
24     fi
25
26     # Ensure correct permissions for /raid1
27     sudo chown -R $(whoami):$(whoami) /raid1
28
29     # Configuration on web1 (primary configuration node)
30     if [[ "$HOSTNAME" == "web1" ]]; then
31         echo "Setting up GlusterFS on web1 (primary configuration node)..."
32
33         # Add web2 as a peer
34         echo "Adding web2 as a peer..."
35         sudo gluster peer probe web2 || echo "web2 is already added as a peer."
36
37         # Create the GlusterFS volumes if they do not exist
38         if ! sudo gluster volume info storage-www &>/dev/null; then
39             echo "Creating GlusterFS volume (storage-www)..."
40             sudo gluster volume create storage-www replica 2 transport tcp web1:/raid1/www web2:/raid1/www force
41             sudo gluster volume start storage-www
42         fi
43
44         if ! sudo gluster volume info storage-sql &>/dev/null; then
45             echo "Creating GlusterFS volume (storage-sql)..."
46             sudo gluster volume create storage-sql replica 2 transport tcp web1:/raid1/sql web2:/raid1/sql force
47             sudo gluster volume start storage-sql
48         fi
49     fi
50
51     # Mount the GlusterFS volume for web directory (on web1 and web2)
52     if [ ! -d "/cluster/www" ]; then
53         echo "Creating /cluster/www directory for mount point..."
54         sudo mkdir -p /cluster/www
55     fi
56
57     if ! grep -q "web1:/storage-www /cluster/www glusterfs" /etc/fstab; then
58         echo "Adding mount for /cluster/www to /etc/fstab..."
59         echo "web1:/storage-www /cluster/www glusterfs defaults,_netdev 0 0" | sudo tee -a /etc/fstab
60     fi
61
62     if ! mount | grep -q "/cluster/www"; then
63         echo "Mounting the GlusterFS volume at /cluster/www..."
64         sudo mount -t glusterfs web1:/storage-www /cluster/www
65     fi
66 fi
```

4.2.3 webserver.sh

Instala e configura o Nginx para servir ficheiros a partir de /cluster/www. Cria um ficheiro index.html de teste para validação.

```
webserver.sh M X
sc-workspace > setup_files > webserver.sh
You, 1 second ago | 1 author (You)
1  #!/bin/bash
2  set -e # Exit script on any error
3
4  echo "Installing and configuring Nginx on $(hostname)..."
5
6  # Install Nginx
7  sudo apt-get install -y nginx
8  # Enable and start Nginx
9  sudo systemctl enable nginx
10 sudo systemctl start nginx
11
12 # Ensure /cluster/www exists
13 > if [ ! -d "/cluster/www" ]; then...
14 fi
15
16 # Set /cluster/www as the Nginx web root (just for now)
17 NGINX_CONF="/etc/nginx/sites-available/default"
18 sudo tee $NGINX_CONF > /dev/null <<EOF
19 server {
20     listen 80;
21     server_name default;
22
23     root /cluster/www;
24     index index.html index.htm;
25
26     location / {
27         try_files $uri $uri/ =404;
28     }
29 }
30 EOF
31 # Reload Nginx to apply the configuration
32 sudo systemctl reload nginx
33
34 # Create a sample index.html file in /cluster/www
35 if [ ! -f "/cluster/www/index.html" ]; then
36     echo "Creating index.html in /cluster/www..."
37     sudo cp /vagrant/index.html /cluster/www/index.html
38 fi
39
40 # Ensure correct permissions
41 sudo chown -R www-data:www-data /cluster/www
42 sudo chmod -R 755 /cluster/www
43
44 echo "Nginx is installed and configured to serve files from /cluster/www on $(hostname)."
```

4.3 Máquinas SQL1 e SQL2

Os servidores sql1 e sql2 foram configurados para executar o **MariaDB** em modo **ativo/passivo**, com os dados armazenados no volume distribuído **storage-sql**, montado em **/cluster/sql** e compartilhado entre os nós via GlusterFS. A configuração foi realizada pelos seguintes scripts:

4.3.1 glusterfs.sh

Garante que o volume storage-sql, criado anteriormente pelos servidores web, esteja montado em /cluster/sql nos nós SQL.

```
glusterfs.sh X
sc-workspace > setup_files > glusterfs.sh
You, last week | 1 author (You)
1  #!/bin/bash
2
3  set -e # Exit script on any error
4
5  # Install GlusterFS
6  echo "Installing GlusterFS..."
7  sudo apt-get install -y glusterfs-server glusterfs-client
8
9  # Start and enable the GlusterFS service
10 sudo systemctl start glusterd
11 sudo systemctl enable glusterd
12
13 # Get the hostname
14 HOSTNAME=$(hostname)
15
16 # Configuration for web nodes (web1 and web2)
17 > if [[ "$HOSTNAME" == "web1" || "$HOSTNAME" == "web2" ]]; then...
66 fi
67
68 # Configuration for SQL nodes (sql1 and sql2)
69 if [[ "$HOSTNAME" == "sql1" || "$HOSTNAME" == "sql2" ]]; then
70     echo "Configuring GlusterFS client on SQL node ($HOSTNAME)..."
71
72     # Mount the GlusterFS volume for SQL directory
73     if [ ! -d "/cluster/sql" ]; then
74         echo "Creating /cluster/sql directory for mount point..."
75         sudo mkdir -p /cluster/sql
76     fi
77
78     if ! grep -q "web1:/storage-sql /cluster/sql glusterfs" /etc/fstab; then
79         echo "Adding mount for /cluster/sql to /etc/fstab..."
80         echo "web1:/storage-sql /cluster/sql glusterfs defaults,_netdev 0 0" | sudo tee -a /etc/fstab
81     fi
82
83     if ! mount | grep -q "/cluster/sql"; then
84         echo "Mounting the GlusterFS volume at /cluster/sql..."
85         sudo mount -t glusterfs web1:/storage-sql /cluster/sql
86     fi
87 fi
88
89 echo "GlusterFS setup completed on node $HOSTNAME." You, last week • gluster modifications
```

4.3.2 database.sh

Instala o MariaDB e configura o diretório de dados para usar /cluster/sql. Inclui ajustes na configuração para associar o MariaDB ao **IP virtual do cluster** (192.168.51.110).

Nota: Caso necessário, o ficheiro completo de configuração está disponível em **/vagrant/conf/50-server.cnf** e pode ser copiado diretamente.

```
database.sh X
sc-workspace > setup_files > database.sh
0
7  # Install MariaDB server
8  sudo apt-get install -y mariadb-server
9
10 # Stop MariaDB service for configuration
11 sudo systemctl stop mariadb
12
13 # Ensure GlusterFS mount exists
14 if [ ! -d "/cluster/sql" ]; then
15     echo "Error: /cluster/sql does not exist. Ensure GlusterFS is properly configured before running this script."
16     exit 1
17 fi
18
```

```

18
19 # Move MariaDB data directory to /cluster/sql
20 if [ -d "/var/lib/mysql" ]; then
21     echo "Relocating MariaDB data directory to /cluster/sql..."
22     if [ "$(ls -A /cluster/sql)" ]; then
23         echo "/cluster/sql already populated. Skipping data migration."
24     else
25         # Migrate data if the directory is empty
26         sudo mv /var/lib/mysql/* /cluster/sql/
27     fi
28     # Remove old directory and create symbolic link
29     sudo rm -rf /var/lib/mysql || true # Avoid stopping the script if the directory does not exist
30     sudo ln -s /cluster/sql /var/lib/mysql
31 fi
32
33
34 sudo chown -R mysql:mysql /cluster/sql
35 sudo chmod -R 755 /cluster/sql
36
37 # Configure MariaDB to use /cluster/sql as the data directory
38 if ! grep -q "datadir = /cluster/sql" /etc/mysql/mariadb.conf.d/50-server.cnf; then
39     echo "Updating MariaDB configuration to use /cluster/sql..."
40     sudo sed -i 's|^datadir.*|datadir = /cluster/sql|' /etc/mysql/mariadb.conf.d/50-server.cnf
41 fi
42
43 # Configure MariaDB to bind to the cluster IP (192.168.51.110)
44 if ! grep -q "bind-address = 192.168.51.110" /etc/mysql/mariadb.conf.d/50-server.cnf; then
45     echo "Updating MariaDB configuration to bind to 192.168.51.110..."
46     sudo sed -i 's|^bind-address.*|bind-address = 192.168.51.110|' /etc/mysql/mariadb.conf.d/50-server.cnf
47 fi
48
49 # Restart MariaDB to apply changes
50 sudo systemctl enable mariadb
51 sudo systemctl start mariadb
52

```

4.3.3 cluster_database.sh

Configura alta disponibilidade para o MariaDB utilizando Pacemaker e Corosync. Inclui:

- Criação de recursos no cluster, nomeadamente o **IP virtual** e o **serviço MariaDB**.
- Estabelecimento de dependências e ordens entre os recursos (MariaDB depende do IP virtual).

```

cluster_database.sh X
sc-workspace > setup_files > cluster_database.sh
8 echo "Installing Pacemaker and Corosync..."
9 sudo apt-get install -y pacemaker corosync
10
11 # Enable and start Pacemaker and Corosync services
12 echo "Enabling and starting Pacemaker and Corosync..."
13 sudo systemctl enable corosync pacemaker
14 sudo systemctl start corosync pacemaker
15
16 # Configure Corosync cluster settings
17 echo "Configuring Corosync cluster settings..."
18 sudo cp /vagrant/conf/db_corosync.conf /etc/corosync/corosync.conf
19 sudo systemctl restart corosync pacemaker
20

```

```

20
21 if [[ "${hostname}" == "sql1" ]]; then
22     echo "Setting up cluster on sql1 using crm..."
23
24     # Disable STONITH
25     echo "Disabling STONITH..."
26     sudo crm configure property stonith-enabled=false
27
28     # Set no-quorum-policy to ignore
29     echo "Configuring cluster properties..."
30     sudo crm configure property no-quorum-policy=ignore
31     sudo crm configure property cluster-infrastructure=corosync
32     sudo crm configure property cluster-name="DBCluster"
33
34     # Create resources
35     echo "Creating resources..."
36
37     # Virtual IP resource
38     sudo crm configure primitive cluster_db_ip ocf:heartbeat:IPaddr2 \
39         params ip=192.168.51.110 cidr_netmask=24 \
40         op monitor interval=5s
41
42     # MariaDB resource
43     sudo crm configure primitive mariadb ocf:heartbeat:mysql \
44         params config="/etc/mysql/mariadb.conf.d/50-server.cnf" datadir="/cluster/sql" \
45         user="mysql" op monitor interval=15s timeout=30s
46
47     # Add colocation: MariaDB depends on VIP
48     sudo crm configure colocation mariadb_with_vip inf: mariadb cluster_db_ip
49
50     # Add order: VIP must start before MariaDB
51     sudo crm configure order start_vip_before_mariadb Mandatory: cluster_db_ip mariadb
52
53     # Verify cluster status
54     echo "Cluster setup complete. Verifying status..."
55     sudo crm status
56

```

Configuração do Corosync (db_corosync.conf)

Define os parâmetros do cluster DBCluster, incluindo os nós sql1 e sql2.

```

20
21 nodelist
22 {
23     node {
24         name: sql1
25         nodeid: 1
26         ring0_addr: 192.168.51.111
27     }
28     node {
29         name: sql2
30         nodeid: 2
31         ring0_addr: 192.168.51.112
32     }
33 }
34

```

4.4 Sistema de Balanceamento de Carga

Os proxies haproxy1 e haproxy2 foram configurados para garantir o balanceamento de carga entre os servidores web e alta disponibilidade. A configuração utilizou os seguintes scripts:

4.4.1 ha_proxy.sh

Instala e configura o HAProxy com o ficheiro de configuração personalizado para balancear o tráfego entre web1 e web2.

```
ha_proxy.sh X
sc-workspace > setup_files > ha_proxy.sh
You, 2 weeks ago | 1 author (You)
1  #!/bin/bash
2
3  set -e # Exit script on any error
4
5  echo "Starting HAProxy setup on $(hostname)..."
6
7  # Install HAProxy
8  echo "Installing HAProxy..."
9  sudo apt-get install -y haproxy
10
11 # Copy the custom HAProxy configuration file
12 echo "Copying custom HAProxy configuration..."
13 sudo cp /vagrant/conf/haproxy.cfg /etc/haproxy/haproxy.cfg
14
15 # Restart and enable HAProxy
16 echo "Restarting and enabling HAProxy service..."
17 sudo systemctl enable haproxy
18 sudo systemctl restart haproxy
19
20 echo "HAProxy setup completed on $(hostname)."
```

Configuração do HAProxy

- Frontend: Recebe as requisições no IP virtual do cluster e porta 80.
- Backend: Distribui o tráfego para web1 e web2 em modo round-robin.

```
haproxy.cfg M X
sc-workspace > setup_files > conf > haproxy.cfg
20 defaults
21
22 # Frontend configuration: Handles incoming traffic
23 frontend http_front
24     bind 172.20.51.1:80 # Virtual IP (Cluster Web) and port 80
25     mode http
26     default_backend web_servers
27
28 # Backend configuration: Redirects traffic to web1 and web2
29 backend web_servers
30     mode http
31     balance roundrobin # Load balancing algorithm
32     server web1 192.168.51.121:80 check
33     server web2 192.168.51.122:80 check
```

4.4.2 cluster_proxy.sh

Configura alta disponibilidade para os proxies com Pacemaker e Corosync. Inclui:

- Criação de um **IP virtual** compartilhado (172.20.51.1).
- Configuração do **HAProxy** como recurso dependente do IP virtual.

```
cluster_proxy.sh X
sc-workspace > setup_files > cluster_proxy.sh
8  echo "Installing Pacemaker and Corosync..."
9  sudo apt-get install -y pacemaker corosync
10
11 # Enable and start Pacemaker and Corosync services
12 echo "Enabling and starting Pacemaker and Corosync..."
13 sudo systemctl enable corosync pacemaker
14 sudo systemctl start corosync
15 sudo systemctl start pacemaker
16
17 # Configure Corosync
18 echo "Configuring Corosync cluster settings..."
19 sudo cp /vagrant/conf/haproxy_corosync.conf /etc/corosync/corosync.conf
20
21 sudo systemctl restart corosync pacemaker
```

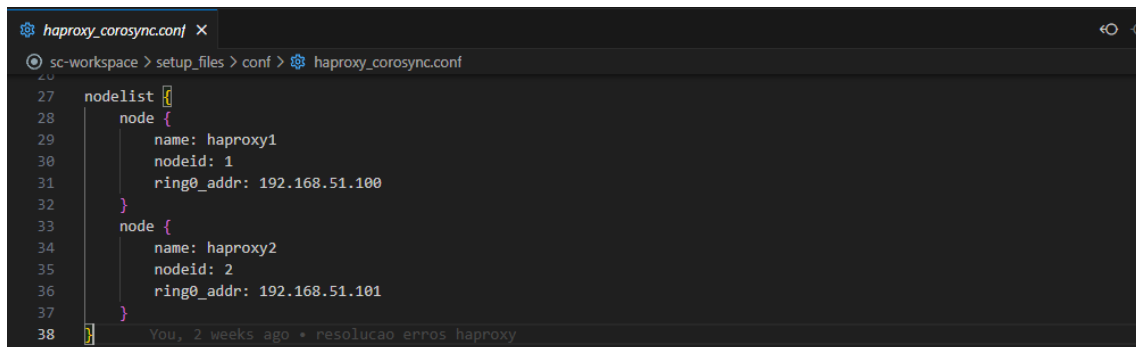


```

22
23 # Additional configuration for haproxy1 (cluster setup)
24 if [[ "$(hostname)" == "haproxy1" ]]; then
25     echo "Setting up cluster on haproxy1 using crm..."
26
27     # Disable STONITH
28     echo "Disabling STONITH and configuring quorum policy..."
29     sudo crm configure property stonith-enabled=false
30     sudo crm configure property no-quorum-policy=ignore
31
32     # Set cluster infrastructure details
33     sudo crm configure property cluster-infrastructure="corosync"
34
35     # Create the cluster
36     sudo crm configure property cluster-name="ProxyCluster"
37
38     # Create resources
39     echo "Creating resources..."
40     sudo crm configure primitive cluster_web_ip ocf:heartbeat:IPaddr2 \
41         params ip=172.20.51.1 cidr_netmask=24 op monitor interval=5s
42     sudo crm configure primitive haproxy systemd:haproxy op monitor interval=5s
43
44     # Add colocation: HAProxy depends on VIP
45     sudo crm configure colocation proxy_with_vip inf: haproxy cluster_web_ip
46
47     # Add order: VIP must start before HAProxy
48     sudo crm configure order start_vip_before_haproxy Mandatory: cluster_web_ip haproxy
49
50     # Verify cluster status
51     echo "Cluster setup complete. Verifying status..."
52     sudo crm status
53 fi
54
55 echo "High availability cluster setup completed on $(hostname)."
```

Configuração do Corosync (haproxy_corosync.conf)

Define os parâmetros do cluster ProxyCluster, incluindo os nós haproxy1 e haproxy2.



```

haproxy_corosync.conf
27 nodelist {
28     node {
29         name: haproxy1
30         nodeid: 1
31         ring0_addr: 192.168.51.100
32     }
33     node {
34         name: haproxy2
35         nodeid: 2
36         ring0_addr: 192.168.51.101
37     }
38 }
```

4.5 Sistema de Ficheiros Distribuído

O sistema de ficheiros distribuído foi configurado com **GlusterFS** para garantir que os dados sejam replicados e compartilhados entre os nós da infraestrutura. Este sistema já foi detalhado nos pontos anteriores, onde foi integrado ao RAID 1 nos servidores web (web1 e web2) e montado nos diretórios necessários para cada funcionalidade:

- **/cluster/www**: Para armazenar e compartilhar os ficheiros da aplicação web.
- **/cluster/sql**: Para armazenar e compartilhar os dados da BD entre sql1 e sql2.

Principais Configurações

Os volumes **storage-www** e **storage-sql** foram criados pelos servidores web1 e web2 utilizando RAID coma base.

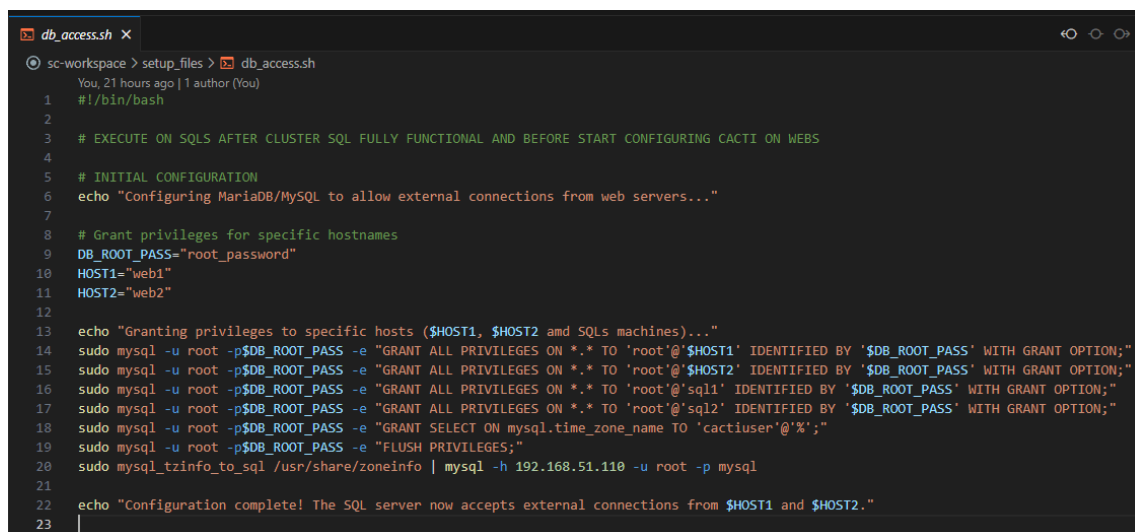
Os nós SQL (sql1 e sql2) montaram o volume **storage-sql**, permitindo o compartilhamento de dados entre eles.

A configuração foi feita com o script **gluster.sh**, já detalhado anteriormente.

4.6 Configuração da Base de Dados para Integração com os Web

Antes de iniciar a preparação e instalação das ferramentas de monitorização no site da empresa, foi necessário configurar a base de dados para permitir acesso externo (acesso a partir das máquinas web) e preparar/antecipar pré-requisitos do Cacti. Esse passo adicional foi realizado com o script **db_access.sh**, executado no nó sql1 após o cluster SQL estar funcional (bem como a infraestrutura como um todo).

Este script ajusta as permissões no MariaDB para permitir conexões dos servidores web (web1 e web2) e configura privilégios específicos para o utilizador usado pelo Cacti.



```
db_access.sh x
sc-workspace > setup_files > db_access.sh
You, 21 hours ago | 1 author (You)
1 #!/bin/bash
2
3 # EXECUTE ON SQLS AFTER CLUSTER SQL FULLY FUNCTIONAL AND BEFORE START CONFIGURING CACTI ON WEBS
4
5 # INITIAL CONFIGURATION
6 echo "Configuring MariaDB/MySQL to allow external connections from web servers..."
7
8 # Grant privileges for specific hostnames
9 DB_ROOT_PASS="root_password"
10 HOST1="web1"
11 HOST2="web2"
12
13 echo "Granting privileges to specific hosts ($HOST1, $HOST2 and SQLs machines)..."
14 sudo mysql -u root -p$DB_ROOT_PASS -e "GRANT ALL PRIVILEGES ON *.* TO 'root'@$HOST1 IDENTIFIED BY '$DB_ROOT_PASS' WITH GRANT OPTION;"
15 sudo mysql -u root -p$DB_ROOT_PASS -e "GRANT ALL PRIVILEGES ON *.* TO 'root'@$HOST2 IDENTIFIED BY '$DB_ROOT_PASS' WITH GRANT OPTION;"
16 sudo mysql -u root -p$DB_ROOT_PASS -e "GRANT ALL PRIVILEGES ON *.* TO 'root'@'sql1' IDENTIFIED BY '$DB_ROOT_PASS' WITH GRANT OPTION;"
17 sudo mysql -u root -p$DB_ROOT_PASS -e "GRANT ALL PRIVILEGES ON *.* TO 'root'@'sql2' IDENTIFIED BY '$DB_ROOT_PASS' WITH GRANT OPTION;"
18 sudo mysql -u root -p$DB_ROOT_PASS -e "GRANT SELECT ON mysql.time_zone_name TO 'cactiuser'@'%';"
19 sudo mysql -u root -p$DB_ROOT_PASS -e "FLUSH PRIVILEGES;"
20 sudo mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -h 192.168.51.110 -u root -p mysql
21
22 echo "Configuration complete! The SQL server now accepts external connections from $HOST1 and $HOST2."
23
```

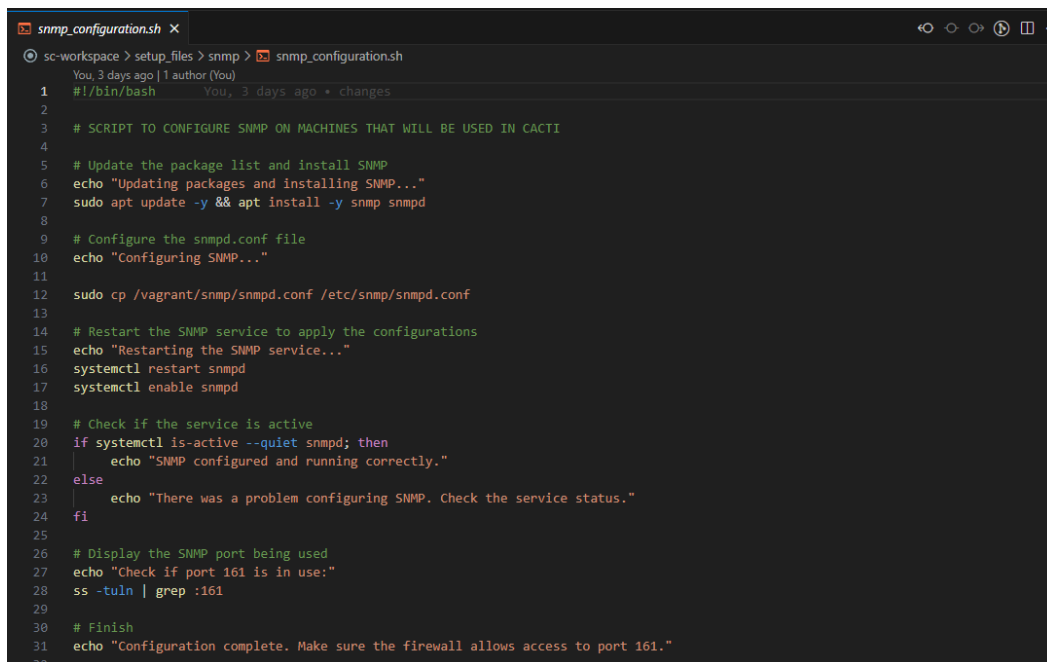
5 SITES DA EMPRESA

5.1 Cacti

Nesta seção, detalho a instalação e configuração do Cacti, uma ferramenta de monitorização configurada para acompanhar os recursos da infraestrutura (CPU, memória, tráfego de rede) através da interface web. Essa instalação foi realizada maioritariamente no nó web1, mas replicada para funcionar em balanceamento com web2.

5.1.1 Preparação SNMP

Primeiro, executei o script **smp_configuration.sh** em todas as máquinas (web1, web2, sql1 e sql2), que instala e configura o serviço SNMP para garantir que as máquinas se registrassem corretamente no sistema de monitoramento do Cacti.



```
snmp_configuration.sh x
sc-workspace > setup_files > snmp > snmp_configuration.sh
You, 3 days ago | 1 author (You)
1  #!/bin/bash      You, 3 days ago * changes
2
3  # SCRIPT TO CONFIGURE SNMP ON MACHINES THAT WILL BE USED IN CACTI
4
5  # Update the package list and install SNMP
6  echo "Updating packages and installing SNMP..."
7  sudo apt update -y && apt install -y snmp snmpd
8
9  # Configure the snmpd.conf file
10 echo "Configuring SNMP..."
11
12 sudo cp /vagrant/snmp/snmpd.conf /etc/snmp/snmpd.conf
13
14 # Restart the SNMP service to apply the configurations
15 echo "Restarting the SNMP service..."
16 systemctl restart snmpd
17 systemctl enable snmpd
18
19 # Check if the service is active
20 if systemctl is-active --quiet snmpd; then
21     echo "SNMP configured and running correctly."
22 else
23     echo "There was a problem configuring SNMP. Check the service status."
24 fi
25
26 # Display the SNMP port being used
27 echo "Check if port 161 is in use:"
28 ss -tuln | grep :161
29
30 # Finish
31 echo "Configuration complete. Make sure the firewall allows access to port 161."
32
```

Faz uso do ficheiro personalizado presente no `/vagrant/snmp/snmpd.conf`.

```
rocommunity public localhost
rocommunity public 192.168.51.0/24
rocommunity public default -V systemonly
```

5.1.2 Preparação e Configuração nos Webs

A instalação e preparação do Cacti foi feita utilizando dois scripts principais, além de referências de configuração externa para ajustes finais.

O script **cacti_prep.sh** realiza as etapas fundamentais para a instalação e configuração inicial do Cacti:

- Instalação dos pacotes necessários (Nginx, PHP, MariaDB Client, SNMP, entre outros).
- Criação da base de dados cacti no cluster SQL e definição do utilizador cactiuser.
- Download e instalação do Cacti na pasta **/cluster/www/cacti**.
- Configuração do cron job para o poller do Cacti.
- Configuração do Nginx para servir o Cacti no endereço <http://172.20.51.1/cacti>.

```
cacti_prep.sh X
sc-workspace > setup_files > cacti > cacti_prep.sh
3 # Initial configuration for Cacti
4 echo "Configuring Cacti..."
5
6 # Configure MariaDB for Cacti
7 DB_ROOT_PASS="root_password"
8 DB_NAME="cacti"
9 DB_USER="cactiuser"
10 DB_USER_PASS="cacti_password"
11 DB_HOST="192.168.51.110"
12
13 # Install required packages
14 sudo apt-get update
15 sudo apt-get install -y nginx php7.4-fpm php7.4-mysql php7.4-snmp php7.4-mbstring mariadb-client snmp snmpd rrdtool
16
17 # Create the Cacti database and user if they don't exist
18 echo "Configuring database for Cacti on the SQL cluster..."
19 DB_EXISTS=$(mysql -h $DB_HOST -u root -p$DB_ROOT_PASS -e "SHOW DATABASES LIKE '$DB_NAME';")
20 if [ -z "$DB_EXISTS" ]; then
21     mysql -h $DB_HOST -u root -p$DB_ROOT_PASS -e "CREATE DATABASE IF NOT EXISTS $DB_NAME;"
22     mysql -h $DB_HOST -u root -p$DB_ROOT_PASS -e "CREATE USER IF NOT EXISTS '$DB_USER'@'%' IDENTIFIED BY '$DB_USER_PASS';"
23     mysql -h $DB_HOST -u root -p$DB_ROOT_PASS -e "GRANT ALL PRIVILEGES ON $DB_NAME.* TO '$DB_USER'@'%' ;"
24     mysql -h $DB_HOST -u root -p$DB_ROOT_PASS -e "FLUSH PRIVILEGES;"
25 else
26     echo "Database $DB_NAME already exists. Skipping creation."
27 fi
28
```

```
cacti_prep.sh M X
sc-workspace > setup_files > cacti > cacti_prep.sh
29 # Install Cacti
30 Cacti_URL="https://www.cacti.net/downloads/cacti-latest.tar.gz"
31 Cacti_DIR="/cluster/www/cacti"
32
33 if [ "$(hostname)" = "web1" ]; then
34     if [ ! -d $Cacti_DIR ]; then
35         echo "Downloading and installing Cacti..."
36         cd /tmp
37         sudo wget $Cacti_URL
38         sudo tar -zxvf cacti-latest.tar.gz
39         sudo rm cacti-latest.tar.gz
40         sudo mv cacti-* $Cacti_DIR
41     else
42         echo "Cacti is already downloaded and extracted."
43     fi
44
45     cd $Cacti_DIR
46
47     # Configure Cacti database settings
48     echo "Configuring Cacti database connection..."
49
50     sudo cp /vagrant/cacti/config.php include/config.php
51
52     # Check if database is already initialized
53     TABLE_COUNT=$(mysql -h $DB_HOST -u $DB_USER -p$DB_USER_PASS -e "SELECT COUNT(*) FROM information_schema.tables WHERE table_schema = '$DB_NAME'"
54     if [ "$TABLE_COUNT" -eq "0" ]; then
55         echo "Importing initial Cacti data into the database..."
56         if [ -f "cacti.sql" ]; then
57             # if not initialized, import the database
58             sudo mysql -h $DB_HOST -u $DB_USER -p$DB_USER_PASS $DB_NAME < cacti.sql
59         else
60             echo "Error: cacti.sql not found in $Cacti_DIR. Cannot initialize database."
61             exit 1
62         fi
63     else
64         echo "Cacti database already initialized. Skipping import and setup."
65     fi
66 else
67     echo "Skipping Cacti setup; this step is only performed on web1."
68 fi
69
```

```
cacti_prep.sh M X
sc-workspace > setup_files > cacti > cacti_prep.sh
71 # Set correct permissions
72 echo "Setting permissions for Cacti files..."
73 sudo chown -R www-data:www-data $Cacti_DIR
74 sudo chmod -R 755 $Cacti_DIR
75
76 # Configure Cron Job for Cacti
77 echo "Configuring cron job for Cacti..."
78 CRON_JOB="*/5 * * * * www-data php /cluster/www/cacti/poller.php > /dev/null 2>&1"
79 CRON_FILE="/etc/crontab"
80
81 if ! grep -Fxq "$CRON_JOB" $CRON_FILE; then
82     echo "$CRON_JOB" | sudo tee -a $CRON_FILE
83 else
84     echo "Cron job already exists in $CRON_FILE. Skipping addition."
85 fi
86
87 # Configure Nginx to serve Cacti
88 echo "Copying Nginx configuration for Cacti..."
89 sudo cp /vagrant/conf/nginx_default.conf /etc/nginx/sites-available/default
90
91 # Activating the Cacti site in Nginx
92 echo "Activating Cacti site in Nginx..."
93 if [ ! -L /etc/nginx/sites-enabled/default ]; then
94     sudo ln -s /etc/nginx/sites-available/default /etc/nginx/sites-enabled/
95 fi
96 sudo systemctl restart nginx
97 # Final message
98 echo "Cacti setup complete! Access: http://172.20.51.1/cacti to complete the installation."
99
```

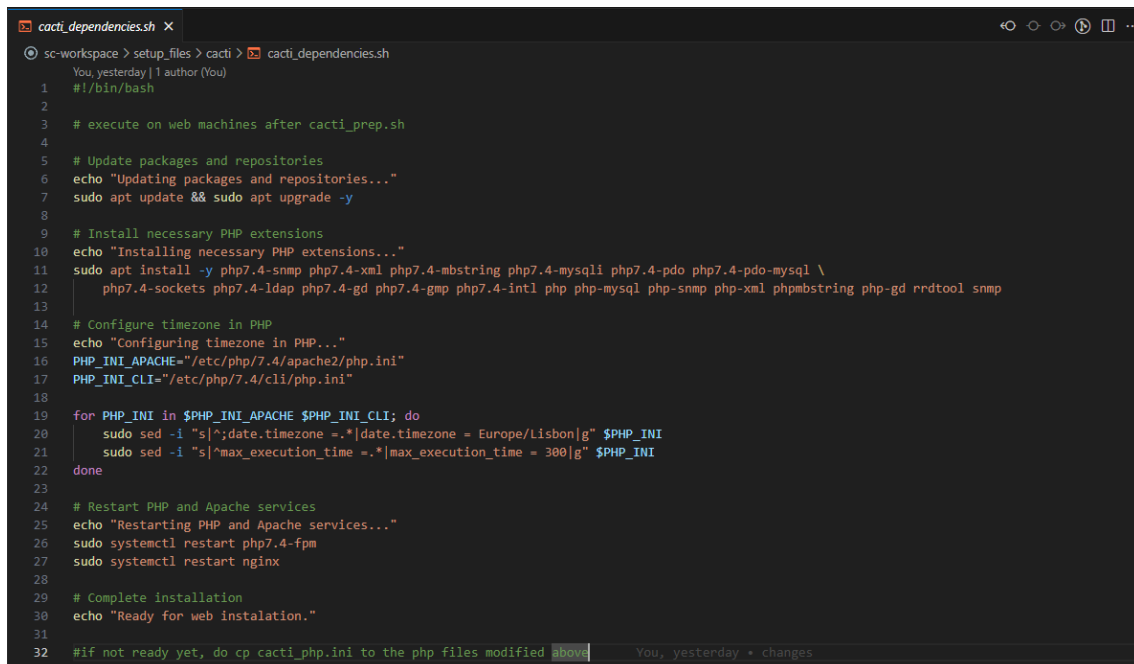
O Cacti foi configurado com o ficheiro `/vagrant/cacti/config.php` que tem de relevante o seguinte trecho:

```
config.php X
sc-workspace > setup_files > cacti > config.php
25 /**
26
27
28
29 $database_type = 'mysql';
30 $database_default = 'cacti';
31 $database_hostname = '192.168.51.110';
32 $database_username = 'cactiuser';
33 $database_password = 'cacti_password';
34 $database_port = '3306';
35 $database_retries = 5;
36 $database_ssl = false;
37 $database_ssl_key = '';
38 $database_ssl_cert = '';
39 $database_ssl_ca = '';
40 $database_persist = false;
41
```

Já o Nginx foi configurado para usar o ficheiro personalizado em `/vagrant/conf/nginx_default.conf`. Nele tem o seguinte:

```
nginx_default.conf X
sc-workspace > setup_files > conf > nginx_default.conf
You, yesterday | 1 author (You)
1 server { You, yesterday * changes
2     listen 80;
3     server_name _;
4
5     # index SC
6     location / {
7         root /cluster/www; # default SC WEB page
8         index index.html;
9     }
10
11     # Cacti Location
12     location /cacti {
13         alias /cluster/www/cacti;
14         index index.php;
15
16         location ~ /\.php$ {
17             include snippets/fastcgi-php.conf;
18             fastcgi_pass unix:/var/run/php/php7.4-fpm.sock;
19             fastcgi_param SCRIPT_FILENAME $request_filename;
20             include fastcgi_params;
21         }
22
23         location ~ /\.ht {
24             deny all;
25         }
26     }
```

O script **cacti_dependencies.sh** garante que as extensões PHP necessárias estão instaladas e configuradas corretamente. Incluiu ajustes importantes no ficheiro `php.ini`, como a definição do fuso horário e o aumento do tempo máximo de execução.



```
sc-workspace > setup_files > cacti > cacti_dependencies.sh
You, yesterday | 1 author (You)
1  #!/bin/bash
2
3  # execute on web machines after cacti_prep.sh
4
5  # Update packages and repositories
6  echo "Updating packages and repositories..."
7  sudo apt update && sudo apt upgrade -y
8
9  # Install necessary PHP extensions
10 echo "Installing necessary PHP extensions..."
11 sudo apt install -y php7.4-snmpp php7.4-xml php7.4-mbstring php7.4-mysqli php7.4-pdo php7.4-pdo-mysql \
12     php7.4-sockets php7.4-ldap php7.4-gd php7.4-gmp php7.4-intl php php-mysql php-snmpp php-xml phpmbstring php-gd rrdtool snmp
13
14 # Configure timezone in PHP
15 echo "Configuring timezone in PHP..."
16 PHP_INI_APACHE="/etc/php/7.4/apache2/php.ini"
17 PHP_INI_CLI="/etc/php/7.4/cli/php.ini"
18
19 for PHP_INI in $PHP_INI_APACHE $PHP_INI_CLI; do
20     sudo sed -i "s|^date.timezone = .*|date.timezone = Europe/Lisbon|g" $PHP_INI
21     sudo sed -i "s|^max_execution_time = .*|max_execution_time = 300|g" $PHP_INI
22 done
23
24 # Restart PHP and Apache services
25 echo "Restarting PHP and Apache services..."
26 sudo systemctl restart php7.4-fpm
27 sudo systemctl restart nginx
28
29 # Complete installation
30 echo "Ready for web installation."
31
32 #if not ready yet, do cp cacti_php.ini to the php files modified above
You, yesterday * changes
```

O ficheiro final de configuração do PHP está disponível em **/vagrant/cacti/cacti_php.ini** e pode ser copiado diretamente se necessário.

Nota: Segui as orientações do guia How to Install Cacti Monitoring Tool on Arch Linux (<https://www.atlantic.net/dedicated-server-hosting/how-to-install-cacti-monitoring-tool-on-arch-linux/>) como referência para algumas etapas.

5.1.3 Configuração via Interface Web

Após todas as configurações automatizadas, finalizei a instalação e ajustes do Cacti pela interface web, acessível em **http://172.20.51.1/cacti**.

Cacti Server v1.2.28 - Installation Wizard

Default Profile

Please select the default Data Source Profile to be used for polling sources. This is the maximum amount of time between scanning devices for information so the lower the polling interval, the more work is placed on the Cacti Server host. Also, select the intended, or configured Cron interval that you wish to use for Data Collection.

Default Profile: 5 Minute Collection

Cron Interval: Every 5 Minutes

Default Automation Network

Cacti can automatically scan the network once installation has completed. This will utilise the network range below to work out the range of IPs that can be scanned. A predefined set of options are defined for scanning which include using both 'public' and 'private' communities.

If your devices require a different set of options to be used first, you may define them below and they will be utilized before the defaults

All options may be adjusted post installation

Default Options

Scan Mode: ☒

Network Range: 192.168.51.0/24

Additional Defaults: ☒

Incluí todas as máquinas no monitoramento e configurei os gráficos principais.

Console | Graphs | Reporting | Logs

Console | Devices | Logged in as admin

Devices

Site: Any | Data Collector: Any | Template: Any | Location: All | Go | Clear | Export

Search: Enter a search term | Status: Any | Devices: Default

Device Description	Hostname	ID	Graphs	Data Sources	Status	In State	Uptime	Poll Time	
Local Linux Machine	localhost	1	4	5	Up	N/A	N/A	0.17	☐
sql1	192.168.51.111	10	5	5	Up	1d:16h:58m	13h:8m	0.08	☐
sql2	192.168.51.112	12	5	5	Up	1d:16h:53m	13h:11m	0.08	☐
web1	192.168.51.121	8	6	6	Up	1d:17h:3m	13h:8m	0.09	☐
web2	192.168.51.122	9	5	5	Up	1d:17h:3m	13h:7m	0.08	☐

All 5 Devices

Choose an action | Go

5.2 Ganglia

A instalação e configuração do Ganglia têm como objetivo a monitorização das máquinas da infraestrutura interna, nomeadamente os servidores web (web1 e web2) e os servidores SQL (sql1 e sql2). Fiz a configuração centralizando os ficheiros de configurações do Ganglia em um diretório compartilhado (/cluster/www/ganglia), para garantir que as configurações de monitoramento sejam consistentes em todos os nós.

5.2.1 Preparação dos Data Sources

Primeiro, executei o script **datasources_prep.sh** em todas as máquinas (web1, web2, sql1 e sql2), que instalou o pacote ganglia-monitor e habilitou o serviço para garantir que as máquinas se registrassem corretamente no sistema de monitoramento do Ganglia.

```
datasources_prep.sh U X
sc-workspace > setup_files > ganglia > datasources_prep.sh
1  #!/bin/bash
2
3  # Script to configure a Data Source for Ganglia
4  # Runned on Web1, web2, sql1 and sql2
5
6  echo "Updating packages..."
7  sudo apt update && sudo apt install -y ganglia-monitor
8  sudo systemctl enable ganglia-monitor
9
10 echo "Data Source preparation complete!"
11
```

5.2.2 Configuração nas Máquinas Web

Em seguida, executei o script **ganglia_web.sh** apenas nas máquinas web (web1 e web2). Este script instalou os pacotes necessários para o Ganglia Web Frontend e assegurou que os ficheiros de configuração personalizados fossem usados corretamente.

```
ganglia_web.sh U X
sc-workspace > setup_files > ganglia > ganglia_web.sh
1  #!/bin/bash
2
3  # NGINX FILE already configured
4  # Runned on web1 and web2 only
5  # Script to configure Ganglia on Web servers
6
7  echo "Updating packages and installing dependencies..."
8  sudo apt update
9  sudo apt install -y ganglia-monitor gmetad ganglia-webfrontend
10 sudo systemctl enable gmetad
11
12 # Check if shared directory /cluster/www/ganglia already exists
13 if [ ! -d "/cluster/www/ganglia" ]; then
14     echo "Preparing shared directory /cluster/www/ganglia..."
15     sudo mkdir -p /cluster/www/ganglia
16     sudo cp -r /usr/share/ganglia-webfrontend/* /cluster/www/ganglia/
17
18     # Configuring gmetad
19     echo "Configuring gmetad.conf..."
20     sudo cp /vagrant/ganglia/gmetad.conf /cluster/www/ganglia/gmetad.conf
21     sudo cp /etc/ganglia/gmond.conf /cluster/www/ganglia/gmond.conf
22
23 > else...
24 fi
25
26 #if symbolic links do not exist, create them
27 if [ ! -L "/etc/ganglia/gmond.conf" ]; then
28     echo "Creating symbolic link for /cluster/www/ganglia/gmond.conf..."
29     sudo rm -f /etc/ganglia/gmond.conf
30     sudo ln -s /cluster/www/ganglia/gmond.conf /etc/ganglia/gmond.conf
31 > else...
32 fi
33
34 if [ ! -L "/etc/ganglia/gmetad.conf" ]; then
35     echo "Creating symbolic link for /cluster/www/ganglia/gmetad.conf..."
36     sudo rm -f /etc/ganglia/gmetad.conf
37     sudo ln -s /cluster/www/ganglia/gmetad.conf /etc/ganglia/gmetad.conf
38 > else...
39 fi
40
41 # Restarting services
42 echo "Restarting Ganglia services..."
43 sudo systemctl restart ganglia-monitor
44 sudo systemctl restart gmetad
45 sudo systemctl restart nginx
46
47 echo "Ganglia configuration complete on $(hostname)!"
48
```

OBS: Os ficheiros gmond.conf e gmetad.conf são configurados como links simbólicos apontando para o diretório compartilhado, garantindo que ambos os servidores web leiam os mesmos arquivos de configuração:

Configuração do gmetad.conf

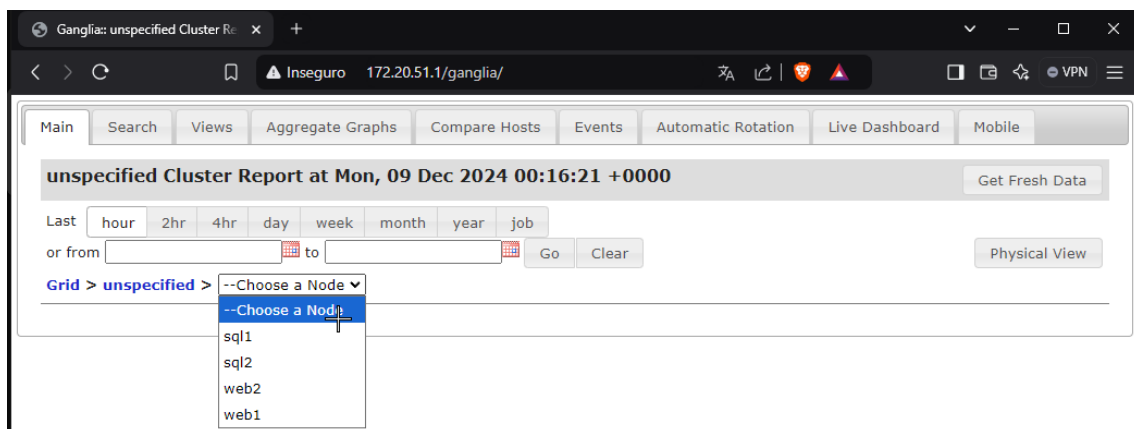
Contém as fontes de dados (data sources) para os servidores monitorados.

```
gmetad.conf U x
sc-workspace > setup_files > ganglia > gmetad.conf
1 data_source "web1" 192.168.51.121:8649
2 data_source "web2" 192.168.51.122:8649
3 data_source "sql1" 192.168.51.111:8649
4 data_source "sql2" 192.168.51.112:8649
```

O nginx já estava configurado para o Ganglia pelo ficheiro nginx_defaults.conf abordado anteriormente. O trecho referente ao ganglia:

```
nginx_default.conf x
sc-workspace > setup_files > conf > nginx_default.conf
1 server {
28 # Ganglia location
29 location /ganglia {
30     alias /cluster/www/ganglia;
31     index index.php index.html index.htm;
32
33     location ~ /\.php$ {
34         include snippets/fastcgi-php.conf;
35         fastcgi_pass unix:/var/run/php/php7.4-fpm.sock;
36         fastcgi_param SCRIPT_FILENAME $request_filename;
37         include fastcgi_params;
38     }
39
40     location ~ /\.ht {
41         deny all;
42     }
43 }
```

O monitoramento já foi possível a partir de <http://172.20.51.1/ganglia>, que agora exibe os dados dos servidores web e SQL.



6 AVALIAÇÃO DA INFRAESTRUTURA

A avaliação da infraestrutura foi realizada para testar a escalabilidade, o desempenho e a robustez da solução implementada. O objetivo principal foi avaliar como a infraestrutura responde ao aumento da carga de utilizadores, especialmente no que diz respeito ao serviço web e aa base de dados, garantindo que o balanceamento de carga e a alta disponibilidade se comportem conforme esperado.

Objetivos da Avaliação:

1. **Escalabilidade do Sistema Web:** Avaliar como a infraestrutura responde ao aumento do número de requisições simultâneas aos servidores web (web1 e web2) e o impacto no desempenho da CPU, memória e nas interfaces de rede.
2. **Impacto na base de Dados:** Testar a performance da base de dados (MariaDB) nos servidores SQL (sql1 e sql2), considerando o aumento de carga nos servidores web e o uso de MariaDB em modo ativo/passivo.
3. **Desempenho do Balanceador de Carga:** Avaliar a capacidade do HAProxy em distribuir o tráfego de forma equilibrada entre os servidores web, além de testar a alta disponibilidade e a tolerância a falhas quando um dos proxies ou servidores falha.
4. **Monitoramento com Cacti/Ganglia:** Utilizar uma das ferramentas Cacti ou Ganglia para monitorar os recursos do sistema (CPU, RAM, tráfego de rede) durante os testes de carga.

Ferramenta de Teste:

Para realizar esses testes, utilizei ferramenta ab (Apache Benchmark), que permite simular requisições simultâneas a um servidor web, fornecendo métricas de desempenho como tempo de resposta, throughput e taxa de erros.

```
ab -n <num de requisições> -c <num de conexões> http://172.20.51.1
```

6.1 Impacto para Serviço Web

Os comandos usados foram:

```
ab -n 25000 -c30 http://172.20.51.1/ganglia
ab -n 25000 -c60 http://172.20.51.1/ganglia
ab -n 25000 -c120 http://172.20.51.1/ganglia
ab -n 25000 -c240 http://172.20.51.1/ganglia
```

Com as duas máquinas Web ativas:

```
vagrant@client:~$ ab -n 25000 -c30 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      30
Time taken for tests:    35.008 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    712.30 [#/sec] (mean)
Time per request:       42.117 [ms] (mean)
Time per request:       1.404 [ms] (mean, across all concurrent requests)
Transfer rate:          267.11 [Kbytes/sec] received
```

```
Connection Times (ms)
  min   mean[+/-sd] median   max
Connect:  0    0.2      0       8
Processing: 1  42  39.8    21    127
Waiting:    1  41  39.8    21    127
Total:      1  42  39.8    24    127
```

Percentage of the requests served within a certain time (ms)

```
50%    24
66%    78
75%    80
80%    81
90%    84
95%    88
98%    97
99%   185
100%  127 (longest request)
```

```
vagrant@client:~$ ab -n 25000 -c60 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      60
Time taken for tests:    35.358 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    707.06 [#/sec] (mean)
Time per request:       84.858 [ms] (mean)
Time per request:       1.414 [ms] (mean, across all concurrent requests)
Transfer rate:          265.15 [Kbytes/sec] received
```

```
Connection Times (ms)
  min   mean[+/-sd] median   max
Connect:  0    0.6      0      18
Processing: 1  84  82.2    35   231
Waiting:    1  84  82.2    34   231
Total:      1  85  82.2    48   232
```

Percentage of the requests served within a certain time (ms)

```
50%    48
66%   161
75%   166
80%   168
90%   174
95%   178
98%   183
99%   187
100%  232 (longest request)
```

```
vagrant@client:~$ ab -n 25000 -c120 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      120
Time taken for tests:    36.031 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    693.85 [#/sec] (mean)
Time per request:       172.948 [ms] (mean)
Time per request:       1.441 [ms] (mean, across all concurrent requests)
Transfer rate:          260.19 [Kbytes/sec] received
```

```
Connection Times (ms)
  min   mean[+/-sd] median   max
Connect:  0    1.6      0  1012
Processing: 1 172 170.6    52   514
Waiting:    1 172 170.6    52   513
Total:      1 172 170.7    58  1014
```

Percentage of the requests served within a certain time (ms)

```
50%    58
66%   326
75%   334
80%   340
90%   357
95%   377
98%   407
99%   428
100% 1014 (longest request)
```

```
vagrant@client:~$ ab -n 25000 -c240 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      240
Time taken for tests:    34.761 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    719.19 [#/sec] (mean)
Time per request:       333.788 [ms] (mean)
Time per request:       1.390 [ms] (mean, across all concurrent requests)
Transfer rate:          269.70 [Kbytes/sec] received
```

```
Connection Times (ms)
  min   mean[+/-sd] median   max
Connect:  0    1.5      0    18
Processing: 1 330 329.2   107  836
Waiting:    1 330 329.2   106  836
Total:      1 331 329.1   119  836
```

Percentage of the requests served within a certain time (ms)

```
50%   119
66%   652
75%   657
80%   660
90%   673
95%   690
98%   728
99%   747
100%  836 (longest request)
```

Com uma máquina WEB:

```
vagrant@client:~$ ab -n 25000 -c30 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      30
Time taken for tests:    14.510 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    1722.95 [#/sec] (mean)
Time per request:       17.412 [ms] (mean)
Time per request:       0.580 [ms] (mean, across all concurrent requests)
Transfer rate:          646.11 [Kbytes/sec] received
```

Connection Times (ms)					
	min	mean[+/-sd]	median	max	
Connect:	0	0	0.4	0	13
Processing:	4	17	4.4	16	68
Waiting:	2	17	4.3	16	67
Total:	5	17	4.4	17	69

Percentage of the requests served within a certain time (ms)	
50%	17
66%	19
75%	20
80%	21
90%	23
95%	25
98%	28
99%	30
100%	69 (longest request)

```
vagrant@client:~$ ab -n 25000 -c120 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      120
Time taken for tests:    16.486 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    1523.82 [#/sec] (mean)
Time per request:       78.750 [ms] (mean)
Time per request:       0.656 [ms] (mean, across all concurrent requests)
Transfer rate:          571.43 [Kbytes/sec] received
```

Connection Times (ms)					
	min	mean[+/-sd]	median	max	
Connect:	0	1	1.7	0	35
Processing:	9	78	14.1	76	149
Waiting:	2	78	14.1	76	149
Total:	34	78	14.0	76	149

Percentage of the requests served within a certain time (ms)	
50%	76
66%	83
75%	88
80%	91
90%	98
95%	103
98%	110
99%	115
100%	149 (longest request)

```
vagrant@client:~$ ab -n 25000 -c60 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      60
Time taken for tests:    16.445 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    1520.23 [#/sec] (mean)
Time per request:       39.468 [ms] (mean)
Time per request:       0.658 [ms] (mean, across all concurrent requests)
Transfer rate:          570.09 [Kbytes/sec] received
```

Connection Times (ms)					
	min	mean[+/-sd]	median	max	
Connect:	0	0	0.2	0	5
Processing:	7	39	8.6	37	88
Waiting:	2	39	8.6	37	88
Total:	7	39	8.6	38	89

Percentage of the requests served within a certain time (ms)	
50%	38
66%	43
75%	45
80%	47
90%	51
95%	54
98%	59
99%	63
100%	89 (longest request)

```
vagrant@client:~$ ab -n 25000 -c240 http://172.20.51.1/ganglia
This is ApacheBench, Version 2.3 <$Revision: 1843412 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests
```

```
Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80
```

```
Document Path:        /ganglia
Document Length:       178 bytes
```

```
Concurrency Level:      240
Time taken for tests:    15.498 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9600000 bytes
HTML transferred:       4450000 bytes
Requests per second:    1613.15 [#/sec] (mean)
Time per request:       148.777 [ms] (mean)
Time per request:       0.620 [ms] (mean, across all concurrent requests)
Transfer rate:          604.93 [Kbytes/sec] received
```

Connection Times (ms)					
	min	mean[+/-sd]	median	max	
Connect:	0	1	1.7	0	20
Processing:	22	148	30.3	145	312
Waiting:	2	147	30.3	144	312
Total:	23	148	30.1	145	313

Percentage of the requests served within a certain time (ms)	
50%	145
66%	159
75%	167
80%	171
90%	185
95%	201
98%	223
99%	236
100%	313 (longest request)

Avaliação de Desempenho

Cenário com Balanceamento de Carga (HAProxy):

- Requisições por segundo: Variou entre 1520 e 1613 requisições/s.
- Tempo total para as requisições: Variou entre 14.5 a 16.4 segundos para 25.000 requisições.
- Tempo por requisição: Em média, de 0.5 a 0.7 ms, considerando todas as requisições simultâneas.

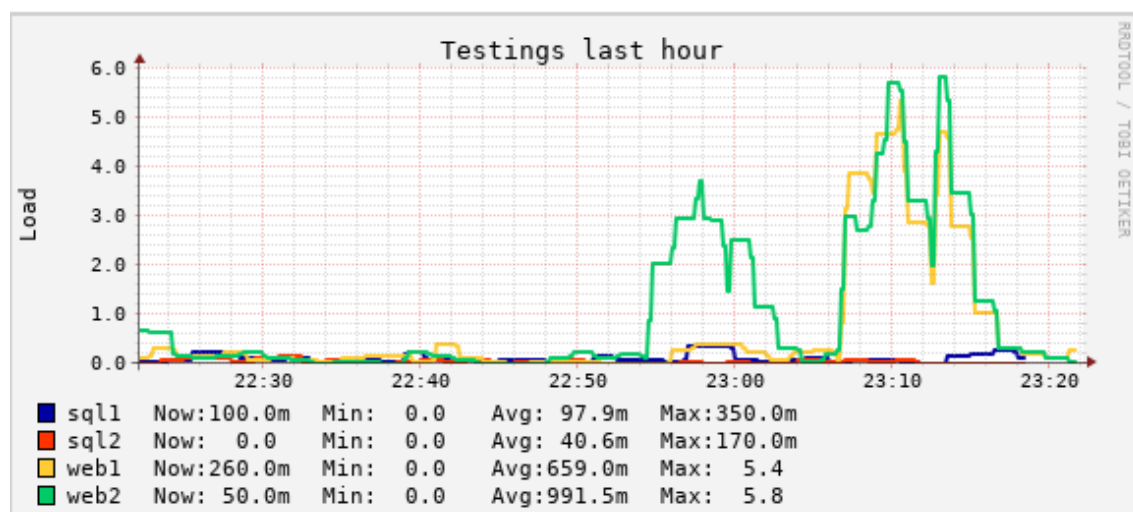
Cenário com uma única Máquina Web (sem Balanceamento de Carga):

- Requisições por segundo: Variou entre 1722 e 1723 requisições/s.
- Tempo total para as requisições: 14.5 segundos para 25.000 requisições.
- Tempo por requisição: 0.58 ms (média para todas as requisições simultâneas).

Análise e Possível Causa da Diferença de Desempenho:

Apesar de o cenário com balanceamento de carga ter mostrado uma leve diminuição na taxa de requisições por segundo, o comportamento observado pode ser explicado por limitações dos próprios recursos em cada máquina, como CPU ou memória. No caso de múltiplas requisições simultâneas (alto nível de concorrência), o balanceador pode ajudar a distribuir a carga, mas pode também adicionar overhead devido à necessidade de comunicação entre os servidores e o gerenciamento de conexões.

Em cenários de alta carga, o balanceamento de carga pode ser vantajoso por evitar que um único servidor se torne sobrecarregado. Portanto, o desempenho pode melhorar com balanceamento em cenários de maior volume de tráfego ou quando um único servidor não é suficiente para lidar com toda a carga.



web1 Host Report at Sun, 08 Dec 2024 23:24:52 +0000

[Get Fresh Data](#)

Last

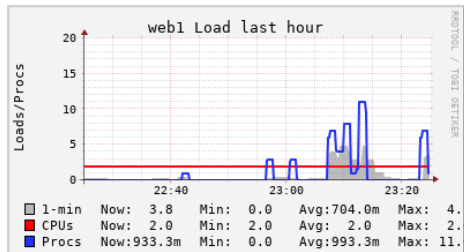
or from to

[Hide/Show Events](#)
[Node View](#)

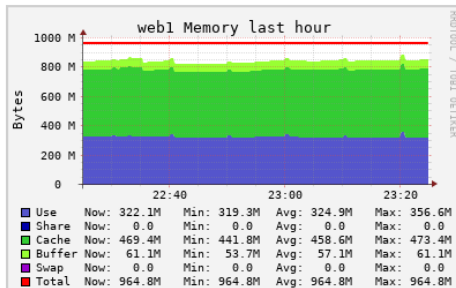
[Grid](#) > [unspecified](#) > [web1](#)

Host Overview

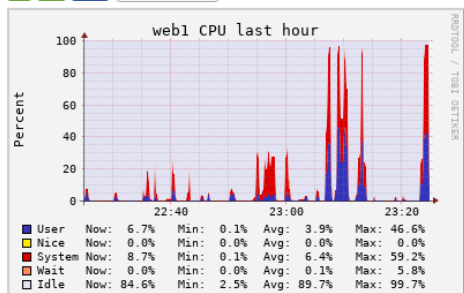
[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



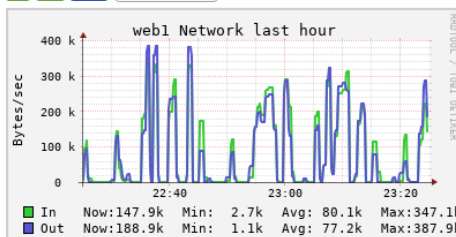
[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



web2 Host Report at Sun, 08 Dec 2024 23:25:22 +0000

[Get Fresh Data](#)

Last

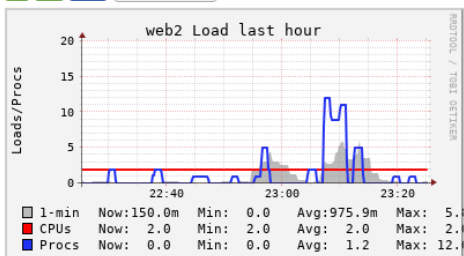
or from to

[Hide/Show Events](#)
[Node View](#)

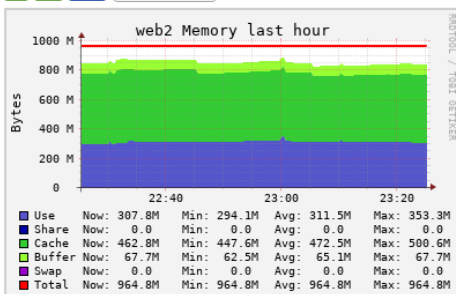
[Grid](#) > [unspecified](#) > [web2](#)

Host Overview

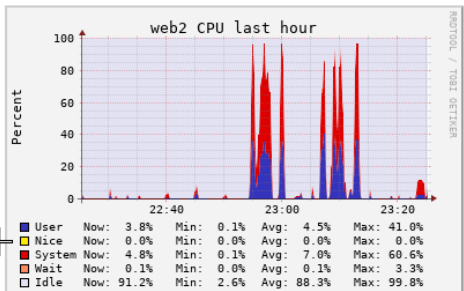
[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



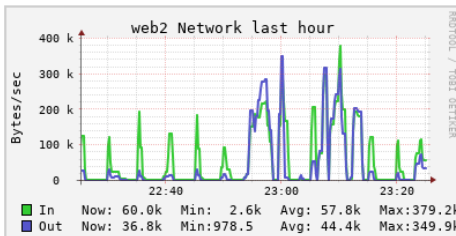
[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



[CSV](#) [JSON](#) [Inspect](#) [Hide/Show Events](#)



6.2 Impacto para Serviço SQL

Para analisar o impacto da ferramenta de pedidos (/cacti) na memória RAM e no CPU do servidor que executa o SGBD MariaDB, continuei realizando testes com o ApacheBench (AB), simulando diferentes níveis de carga na aplicação. Os testes foram executados nas condições anteriores.

Os comandos utilizados foram os seguintes:

```
ab -n 25000 -c30 http://172.20.51.1/cacti
ab -n 25000 -c60 http://172.20.51.1/cacti
ab -n 25000 -c120 http://172.20.51.1/cacti
ab -n 25000 -c240 http://172.20.51.1/cacti
```

```
vagrant@client:~$ ab -n 25000 -c30 http://172.20.51.1/cacti
This is ApacheBench, Version 2.3 <Revision: 1843412 >
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests

Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80

Document Path:        /cacti
Document Length:       178 bytes

Concurrency Level:    30
Time taken for tests:  49.902 seconds
Complete requests:    25000
Failed requests:       0
Non-2xx responses:    25000
Total transferred:    9550000 bytes
HTML transferred:     4450000 bytes
Requests per second:  500.08 [#/sec] (mean)
Time per request:     99.882 [ms] (mean)
Time per request:     1.996 [ms] (mean, across all concurrent requests)
Transfer rate:        186.89 [Kbytes/sec] received

Connection Times (ms)
  min  mean[+/-sd] median  max
Connect:  0    0  0.2    0    9
Processing:  3  59  33.4   62  147
Waiting:    3  59  33.4   62  147
Total:      3  60  33.4   62  148

Percentage of the requests served within a certain time (ms)
 50%    62
 66%    83
 75%    91
 80%    95
 90%   103
 95%   107
 98%   112
 99%   116
100%   148 (longest request)

vagrant@client:~$ ab -n 25000 -c60 http://172.20.51.1/cacti
This is ApacheBench, Version 2.3 <Revision: 1843412 >
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests

Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80

Document Path:        /cacti
Document Length:       178 bytes

Concurrency Level:    60
Time taken for tests:  51.857 seconds
Complete requests:    25000
Failed requests:       0
Non-2xx responses:    25000
Total transferred:    9550000 bytes
HTML transferred:     4450000 bytes
Requests per second:  482.10 [#/sec] (mean)
Time per request:     124.456 [ms] (mean)
Time per request:     2.074 [ms] (mean, across all concurrent requests)
Transfer rate:        179.84 [Kbytes/sec] received

Connection Times (ms)
  min  mean[+/-sd] median  max
Connect:  0    0  0.2    0    4
Processing:  2  124  98.7  125  441
Waiting:    2  124  98.7  125  440
Total:      3  124  98.7  126  441

Percentage of the requests served within a certain time (ms)
 50%   126
 66%   209
 75%   218
 80%   222
 90%   231
 95%   241
 98%   288
 99%   341
100%   441 (longest request)
```



```
vagrant@client:~$ ab -n 25000 -c120 http://172.20.51.1/cacti
This is ApacheBench, Version 2.3 <Revision: 1843412>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests

Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80

Document Path:        /cacti
Document Length:       178 bytes

Concurrency Level:     120
Time taken for tests:   51.574 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9550000 bytes
HTML transferred:       4450000 bytes
Requests per second:    484.74 [#/sec] (mean)
Time per request:       247.554 [ms] (mean)
Time per request:       2.063 [ms] (mean, across all concurrent requests)
Transfer rate:          180.83 [Kbytes/sec] received

Connection Times (ms)
  min  mean[+/-sd] median   max
Connect:  0    0  1.0    0    130
Processing:  3  246 205.1   246   620
Waiting:    3  246 205.1   246   620
Total:      3  246 205.1   247   721

Percentage of the requests served within a certain time (ms)
 50%    247
 66%    444
 75%    460
 80%    467
 90%    483
 95%    500
 98%    534
 99%    556
100%    721 (longest request)

vagrant@client:~$ ab -n 25000 -c240 http://172.20.51.1/cacti
This is ApacheBench, Version 2.3 <Revision: 1843412>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.20.51.1 (be patient)
Completed 2500 requests
Completed 5000 requests
Completed 7500 requests
Completed 10000 requests
Completed 12500 requests
Completed 15000 requests
Completed 17500 requests
Completed 20000 requests
Completed 22500 requests
Completed 25000 requests
Finished 25000 requests

Server Software:      nginx/1.18.0
Server Hostname:      172.20.51.1
Server Port:          80

Document Path:        /cacti
Document Length:       178 bytes

Concurrency Level:     240
Time taken for tests:   50.837 seconds
Complete requests:      25000
Failed requests:         0
Non-2xx responses:      25000
Total transferred:      9550000 bytes
HTML transferred:       4450000 bytes
Requests per second:    491.77 [#/sec] (mean)
Time per request:       488.032 [ms] (mean)
Time per request:       2.033 [ms] (mean, across all concurrent requests)
Transfer rate:          183.45 [Kbytes/sec] received

Connection Times (ms)
  min  mean[+/-sd] median   max
Connect:  0    1  1.5    0    21
Processing: 24  484 195.5   505   786
Waiting:    8  484 195.5   504   786
Total:     30  485 195.3   505   786

Percentage of the requests served within a certain time (ms)
 50%    505
 66%    655
 75%    683
 80%    697
 90%    725
 95%    745
 98%    757
 99%    762
100%    786 (longest request)
```

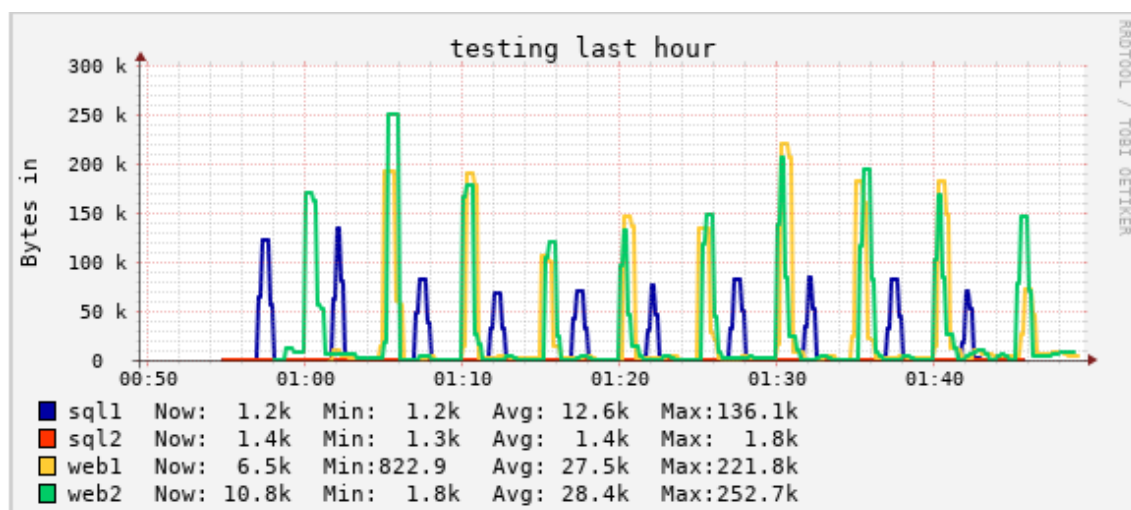
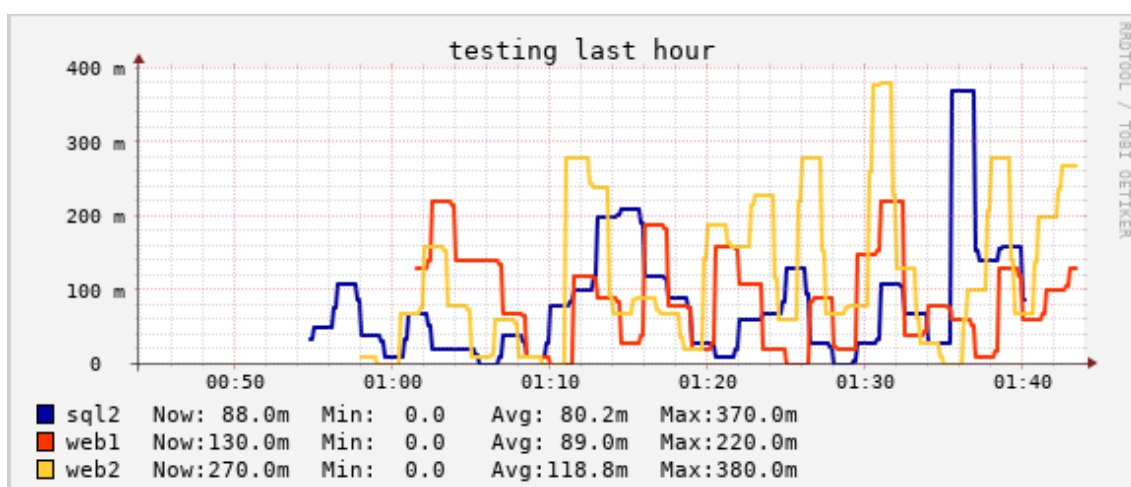
Foram realizadas requisições com diferentes níveis de concorrência (30, 60, 120, 240 conexões simultâneas). Os resultados obtidos em termos de desempenho foram os seguintes:

- **Com 30 conexões simultâneas:**
 - Requisições por segundo: 500.98
 - Tempo total: 49.9 segundos
 - Tempo médio por requisição: 59.88 ms
- **Com 60 conexões simultâneas:**
 - Requisições por segundo: 482.10
 - Tempo total: 51.86 segundos
 - Tempo médio por requisição: 124.46 ms
- **Com 120 conexões simultâneas:**
 - Requisições por segundo: 484.74
 - Tempo total: 51.57 segundos
 - Tempo médio por requisição: 247.55 ms
- **Com 240 conexões simultâneas:**
 - Requisições por segundo: 491.77
 - Tempo total: 50.84 segundos
 - Tempo médio por requisição: 488.03 ms

Análise

Com o aumento no número de conexões simultâneas, observamos uma queda no número de requisições por segundo e um aumento no tempo médio por requisição. Isso é esperado, pois com mais conexões, a máquina ativa (executando MariaDB) precisa processar mais pedidos simultaneamente, o que pode impactar o uso de CPU e memória.

No entanto, em cenários com maior número de requisições, o impacto da base de dados na performance pode ser atenuado com a distribuição de carga entre os servidores web, otimizando o processamento de requisições e evitando sobrecarga em um único servidor.



7 CONCLUSÃO

O objetivo principal deste trabalho foi a implementação de uma infraestrutura de alta disponibilidade para sistemas críticos, com ênfase no balanceamento de carga, redundância de servidores e armazenamento compartilhado. A solução foi baseada em tecnologias como HAProxy, MariaDB com replicação ativa/passiva, e GlusterFS para garantir escalabilidade, resiliência e desempenho da arquitetura.

Durante a execução do projeto, todos os objetivos principais foram cumpridos: a infraestrutura foi configurada corretamente, os servidores web e de banco de dados operam com redundância, e o balanceamento de carga foi implementado de forma eficaz. Testes foram realizados para garantir que a arquitetura estivesse apta a lidar com falhas, com a recuperação automática de servidores e a distribuição de tráfego entre os nós ativos, atendendo aos requisitos de alta disponibilidade. A configuração de armazenamento compartilhado via GlusterFS também contribuiu para a consistência e a redundância dos dados.

Contudo, alguns limites foram encontrados durante a fase de testes. O impacto dos testes de carga no sistema não pôde ser avaliado de forma mais aprofundada devido a conflitos entre as máquinas e problemas de monitoramento relacionados ao Ganglia. Questões como corrupção de dados ou falhas na atualização dos dados de monitoramento prejudicaram a análise precisa dos resultados. Consequentemente, a análise qualitativa dos resultados, com a criação de tabelas ou gráficos que permitissem uma visualização clara da evolução do desempenho do serviço, não foi possível.

Apesar desses desafios, a infraestrutura funcionou de maneira sólida, sendo capaz de lidar com falhas simuladas, com recuperação automática e sem comprometer a disponibilidade do sistema. As ferramentas de monitoramento implementadas forneceram uma visão geral da operação do sistema, embora a avaliação detalhada do impacto de carga não tenha sido realizada conforme planejado.

Em resumo, o trabalho alcançou com sucesso a implementação da alta disponibilidade, cumprindo a maioria dos objetivos estabelecidos, e a infraestrutura criada está pronta para ser utilizada em um ambiente de produção, com potencial para ser expandida e aprimorada com base nas lições aprendidas durante o desenvolvimento. A falta de uma avaliação mais profunda dos resultados de desempenho não compromete o sucesso geral do projeto, mas sugere áreas de melhoria, especialmente em termos de estabilidade de ferramentas de monitoramento e a integração de testes de carga.